

From the desks of ROS maintainers: A survey of modern & capable mobile robotics algorithms in the robot operating system 2

Steve Macenski^{a,*}, Tom Moore^{b,2}, David V. Lu^{c,3}, Alexey Merzlyakov^{d,4}, Michael Ferguson^{e,3}

^a Samsung Research, United States of America

^b Locus Robotics, United States of America

^c Metro Robots, United States of America

^d Samsung Research, Russia

^e Cobalt Robotics, United States of America

ARTICLE INFO

Article history:

Received 14 November 2022

Received in revised form 16 May 2023

Accepted 17 July 2023

Available online 27 July 2023

Keywords:

Navigation

Robot operating system

Nav2

Mobile robotics

Planning

Costmap

ROS

Control

Localization

Perception

SLAM

State estimation

ABSTRACT

The Robot Operating System 2 (ROS 2) is rapidly impacting the intelligent machines sector — on space missions, large agriculture equipment, multi-robot fleets, and more. Its success derives from its focused design and improved capabilities targeting product-grade and modern robotic systems. Following ROS 2's example, the mobile robotics ecosystem has been fully redesigned based on the transformed needs of modern robots and is experiencing active development not seen since its inception.

This paper comes from the desks of the key ROS Navigation maintainers to review and analyze the state of the art of robotics navigation in ROS 2. This includes new systems without parallel in ROS 1 or other similar mobile robotics frameworks. We discuss current research products and historically robust methods that provide differing behaviors and support for most every robot type. This survey consists of overviews, comparisons, and expert insights organized by the fundamental problems in the field. Some of these implementations have yet to be described in literature and many have not been benchmarked relative to others. We end by providing a glimpse into the future of the ROS 2 mobile robotics ecosystem.

© 2023 Elsevier B.V. All rights reserved.

1. Introduction

The Robot Operating System 2 (ROS 2) was redesigned from the ground up to meet the requirements of modern robotics research and commercial products in warehouses, urban settings, agriculture, aerospace, autonomous driving, and more. It also provides previously missing capabilities such as security, multi-robot communications, and embedded/real-time support. Philosophically, ROS 2 was created as a production-grade robotics framework to serve the growing and maturing robotics industry [1].

* Corresponding author.

E-mail addresses: s.macenski@samsung.com (S. Macenski), tmoores@locusrobotics.com (T. Moore), davidvlu@gmail.com (D.V. Lu), alexey.merzlyakov@samsung.com (A. Merzlyakov), fergs@cobaltrobotics.com (M. Ferguson).

¹ ROS 2 Project Lead.

² Robot Localization Maintainer.

³ ROS 1 Co-Maintainer.

⁴ ROS 2 Developer.

The transition to ROS 2 has sparked a renaissance of new development in long-dormant project ecosystems. This new development is holistic, comprising algorithmic refreshes, new features, documentation, and increased maturity inspired by ROS 2's philosophical and procedural changes. The mobile robotics ecosystem in ROS 2 has seized this opportunity to rethink and redesign based on the needs of modern systems.

Nav2 is the major project within the mobile robotics ecosystem and provides a next-generation autonomous mobile robotics navigation framework and system [2]. The original Navigation Stack was developed at Willow Garage for the PR2 robot and based around an unconfigurable state machine, `move_base` [3]. It largely supported circular differential-drive and omnidirectional research platforms.

Nav2 was built from the ground up to leverage innovations in behavior trees and contains refreshed algorithms. It supports all common forms of classical robots (e.g. differential, omnidirectional) as well as newer robot platforms such as Ackermann, large, non-circular, and quadruped robots. It maintains

production-grade quality standards with 90% unit test coverage and seeks to re-centralize development across academia and industry.

The Nav2 project and ROS 2's mobile robotics ecosystem have been developing new capabilities at a pace not seen since the days of Willow Garage. New algorithms, features, and behavior trees are being added on a quarterly basis as the result of open collaborations between companies, individuals, and academia. Over the past two years, a compelling set of new algorithms and behaviors have been made available – some of which have no comparable capability in other similar frameworks (ROS 1, MRPT, etc.) – and have yet to be compared in literature [4]. The introduction of Nav2 [2] showcases many of these new algorithms and behaviors working on commercial robots in highly dynamic environments navigating over 2 marathons in length on an active college campus. The authors recommend reviewing this document for a practical and reproducible example of the composed capabilities described in this paper.

This survey comes from the desks of core ROS developers and maintainers to be the authoritative discussion on the state-of-the-art in ROS 2 robotics navigation. This survey includes overviews, analysis, and unique expert insight into the algorithms and capabilities, new and old. We discuss and compare current research products with historically robust methods. Brief benchmarking within provides metrics for the first time for the new key methods. Finally, this survey ends with a glimpse into the future of ROS 2 mobile robotics capabilities.

2. Overview of global path planners

Global path planning seeks to address the foundational problem of finding optimal sequences of valid configurations to determine a route through an environment. A path may have different levels of fidelity. It may comprise sequences of neighboring cells in a grid map, feasible and continuous sequences, or even vehicle dynamics. For global path planning in Nav2, only up to kinematic constraints are considered. The hybrid planning schema utilized establishes that the global planner's objective is to find a route through the environment, while a local trajectory planner follows this route considering system dynamics and other criteria [5].

The definition of optimal in global path planning can vary with application, and at its core, the algorithms are all extensions of Dijkstra's algorithm in trying to find the path with minimal cost [6]. In Nav2, the cost is largely determined by the *cost map* which is discussed at length in Section 5. In practice, the result is usually the shortest path that does not navigate too closely to any obstacle, as seen in Fig. 1.

Path planners are typically either search- or sampling-based. Sampling-based planners, like RRT and its variants, are prevalent for higher dimensional planning problems and can provide near-optimal paths – although they commonly require post-processing before use [7]. However, search-based planners in lower dimensional state spaces, like those found in mobile robotics, are typically faster and produce truly optimal paths when heuristics are admissible and consistent. Search-based planners also provide more predictable execution times important to dynamic replanning over complex large-scale environments.

Thus, Nav2 provides primarily search-based global planning methods. The furnished algorithms cover the full spanning set of both circular and arbitrarily-shaped robot models, including differential-drive, holonomic, Ackermann, and legged robot platforms. Namely, these are Navigation Function, 2D-A*, Theta*, Hybrid-A*, and State Lattice planners. All of the planners are cost-aware, meaning they consider the range of cost map risk values populated from sensor data, perception pipelines, or semantic information such as maps – not solely binary obstacles.

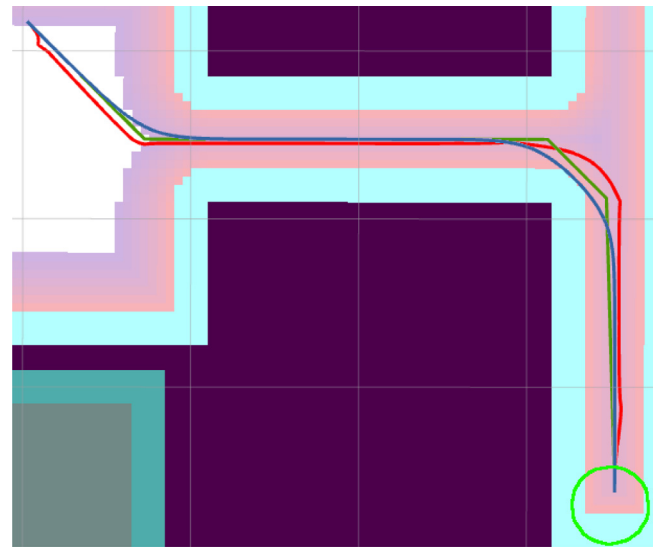


Fig. 1. Example plans from the holonomic planners. NavFn in red, Theta* in green, 2D-A* in blue. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

For comparison, a AMD Ryzen 5 5600X CPU with Ubuntu 20.04 was utilized to benchmark each planning algorithm. The experiment contained 1000 randomly generated start and goal poses through a 20% randomly occupied 10,000 m² map. Each start and goal pairing was made to be at least 3 m apart and known to be reachable. All of the algorithms are analyzed with the same 1000 paths and uses the provided default settings. The results are shown in Table 1.

2.1. Holonomic planners

Nav2 provides a variety of holonomic path planners. This class of planner is most useful for omni-directional or differential-drive robots, particularly those that may be approximated as circular. Non-circular robots that are small relative to their environment may be treated as circular for global planning purposes if there is no navigable corridor too narrow to accommodate an in-place rotation. While differential-drive robots cannot move at any angle, they may perform in-place rotations to a given requested heading if required. Thus, many forms of differential-drive platforms may effectively use holonomic planners.

These planners generally create paths via traditional grid-based algorithms. Since they do not track robot orientation during grid-search, these algorithms collision check using point-costs of the visited cells in an inflated or convolved cost grid, which are discussed further in Section 5. Those included in Nav2 are Navigation Function (commonly known as *NavFn*), Theta*, and smoothed 2D-A* [8–10].

Navigation Function planner is a modified Global Dynamic Window Approach planner based on the NF1 function [8]. It uses Dijkstra's Algorithm to expand a potential field from the goal cell to the start using a Von Neumann neighborhood. The potential field is calculated using a quadratic kernel function rather than distances to the goal (NF1) to create a smoother potential field, and thus smoother output paths [11]. The kernel at a cell is found based on the cell's cost and the value of its neighboring cells' potentials. The planning costs at each cell are linearly remapping from the cost grid to produce a minimum cost to travel, or neutral cost. Therefore, each cell's cost ranges from the neutral to maximum cost when populating the potential field. This generates increasing costs each wavefront until the start cell

Table 1

Path planners comparison, Bolded within 3% of best.

Path planner	Plan time (ms)	Path length (m)	Feasible	Circular	Non-Circ.	Diff.	Omni.	Ackermann	Legged
NavFn	61.02	52.25	–	✓	–	✓	✓	–	–
Lazy Theta*-P ^a	94.42	50.28	–	✓	–	✓	✓	–	–
Smac 2D-A* ^a	88.82	49.65	–	✓	–	✓	✓	–	–
Smac Hybrid-A* ^a	38.77	50.78	✓	✓	✓	✓	–	✓	✓
Smac State Lattice ^a	39.40	50.51	✓	✓	✓	✓	✓	✓	✓

An inter-modal algorithm selection summary can be found in [Appendix A](#).^aExclusive to ROS 2.

is found. Finally, to traceback the path, gradient descent is applied from the start cell in the potential field in half resolution sized increments to produce a uniformly distributed path.

This planner is the oldest algorithm in Nav2 – derived from the ROS (1) Navigation Stack and created in the first public commit to the project. Regardless of age, it is highly optimized with managed priority buffers and remains as one of the most effective holonomic planners available, typically taking between 15–175 ms. It has been proven by dozens, if not hundreds, of robotics companies and researchers over the past ten years in a variety of industry, consumer, and service applications. Due to its speed and long-term stability, the Navigation Function planner is a good general-purpose option and remains the default planning algorithm in Nav2.

Although it is the fastest holonomic planner, it produces paths 5% longer by design, [Table 1](#). The neutral cost balances cell costs with path lengths when generating the potential field. It was carefully selected empirically to produce high-quality paths in a variety of environments, including routing down centers of hallways, maintaining safe margins from obstacles when space is allotted, and lacking in excessive movements. The planner is made to prefer slightly longer routes to minimize cost, which provides additional margins from collision, which can be seen on [Fig. 1](#). However, the neutral cost may be tuned for different behaviors.

Lazy Theta*-P is an any-angle path planner based on A* [9]. The base Theta* technique modifies A* with line-of-sight (LoS) checks between the parent of a node and the neighbor being expanded from that node. If there is LoS, that connection is considered to short-cut the current visited node. In this implementation, each grid cell is a potential search node. During planning, the visited nodes may be repeatedly short-cutted until there is no longer a valid line-of-sight – resulting in the possibility of any-angle LoS between potentially distant parent and child nodes.

Line-of-sight checks are implemented using Bresenham's ray-casting algorithm [12]. The LoS check evaluates if there is a collision with the environment between the two nodes; but the Theta* planner in Nav2 also evaluates if the integrated cost of the line is below a configurable threshold to be considered valid. This process breaks LoS for a given plan heading to be more responsive to the localized environment in high-cost (e.g. confined) areas. In practice, this makes the planner cost-aware, similar to other planners discussed, such that a path does not route unacceptably close to obstacles. This threshold may be tuned for an optimal trade-off between segment lengths and proximity to obstacles. After planning, the path is interpolated over its constituent LoS segments to create a regularized path.

Nav2's Lazy Theta*-P uses a Moore neighborhood and L2 distance heuristic. Lazy Theta* optimizes the algorithm by reducing the number of line-of-sight checks by delaying them until they are strictly necessary [13]. This lazy evaluation conducts only one LoS check when visiting a cell rather than for each expanded cell, yielding a reduction in expensive LoS checks. Lazy Theta*-P builds on this by evaluating LoS on the current node's grandparent rather than parent in the graph. Lazy Theta*-P also has the

important property that every expanded cell will have definite line-of-sight with its set parent. The speculative assumption in Lazy Theta* that a line-of-sight between the node's neighbor and parent exists is not always true. Even if this is corrected when visited, it does not maintain the same properties as A*. However, as a consequence, Lazy Theta*-P performs more LoS checks than Lazy Theta*, as it needs to expand more cells.

Since Theta* and its variants may produce plans at any angle, it is particularly useful for maps at non-axially aligned angles where planners restricted to 4- or 8-connected neighborhoods would produce jagged paths. While it is common to rotate maps such that they are axially aligned with important features, that is not always practical nor possible. It is also useful in spaces with many long, straight corridors commonly found in office buildings, warehouses, retail stores, and similar. Theta*'s straight line characteristics are powerful for predictability of robot behavior to bystanders, as well as providing more fine-tuned and localized alignment to the environment, [Fig. 1](#).

In experiments, this algorithm had the shortest path lengths, yet the longest run-time due to the expensive LoS checks. Yet it still displays appropriate run-time performance under 100 ms making it applicable in many practical applications, [Table 1](#).

2D-A* planner is the simplest of the holonomic approaches, as a cost grid-A* planner [10]. It is implemented within the *Smac Planner* Framework complimentary to the feasible planners discussed in Section 2.2. See [Appendix B](#) for more information about the *Smac Planner* Framework. The 2D-A* planner uses Moore neighborhoods with an L2 distance heuristic. The travel cost between cells is derivative of the cost-aware obstacle heuristic discussed later in detail, and takes the form:

$$cost_{travel} = d \left(1.0 + \omega_{cost} \frac{cost(x, y)}{cost_{max}} \right) \quad (1)$$

where d is the distance between cells, ω_{cost} is the configurable weight placed on penalizing cost, $cost(x, y)$ is the cost at a cell (x, y) , and $cost_{max}$ is the maximum possible cost to normalize the term relative to the penalty weight. When the cost is 0, it is simply the distance between cells (1.0 or $\sqrt{2}$), thereby the heuristic is admissible.

To smooth out the path generated from backtracing cells, the simple smoothing algorithm from Eq. (4) is employed to remove jagged edges, smooth turns, and eliminate other localized defects. This planner uniquely enables users to downsample cost grids into different resolutions for holonomic planning to increase speed $O(N^2)$ in large spaces.⁵ Unsurprisingly, since the *Smac Planners* share much of the search and penalty structures, their paths are within 3% of each other ([Table 1](#)), where 2D-A* is the shortest largely attributable to the lack of non-holonomic constraints.

The 2D-A* planner is most useful when being deployed in heterogeneous fleets of robots, whereas the non-holonomic or large robots are using other planners within the *Smac Planner* framework. This enables all robots in a fleet to have consistent planning behavior based on analogous penalty functions trading off grid map cost and path length using the same weights.

⁵ Downsampled grids of half resolution results in planning times competitive with the other *Smac Planner* implementations.

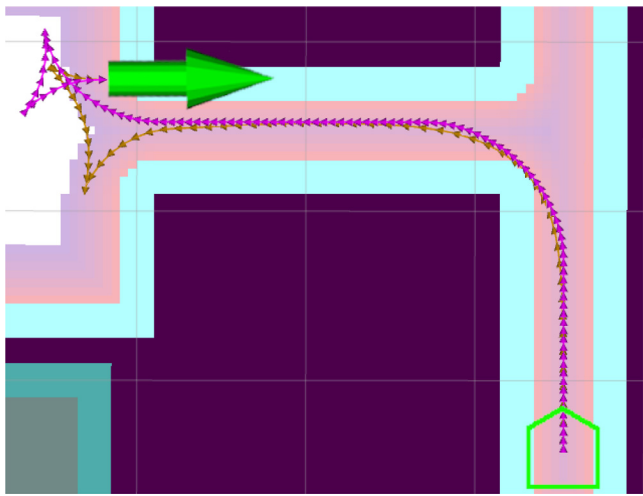


Fig. 2. Example plans for kinematically feasible planners with reversing. Hybrid-A* in brown, State Lattice in magenta. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

2.2. Kinematically feasible planners

In contrast to the holonomic planners, Nav2 also provides a set of kinematically feasible planners – or planners that consider kinematic constraints. These planners enable integration with modern robots such as Ackermann and legged robots, as well as providing accurate modeling of non-circular differential-drive and holonomic bases. There are many cases where classical robots are large relative to their environment and require explicit modeling to create feasible global paths. Since orientations are being considered, collision checking and planning may use the robot shape to generate paths that are high fidelity even through narrow spaces. The feasible planners in Nav2 are Hybrid-A* and State Lattice, both members of the *Smac Planner* framework.

Hybrid-A* was proposed by Stanford during the DARPA Urban Challenge as an extension of traditional grid-A* [14]. It uses motion primitives to search a grid map in a tree-fashion using continuous coordinates associated at discrete cells (x, y, θ) . This ensures continuous and feasible paths. The primitives are composed of straight segments and circular arcs (left and right) of the minimal turning radius for a car-like base. Periodic analytic expansions are applied to find exact and optimal solutions from a currently expanded pose to the goal pose. If this expansion is valid and collision free, it is included in the route to complete the planning process. The search primitives and analytic expansions can be computed utilizing using Dubin or Reeds-Shepp models, depending if reversing is a permissible action.

Thus, this method is specifically designed for car-like or Ackermann vehicles; however, it has utility for legged and large robots beyond naked feasibility of non-circular platforms. Limiting path curvature can be of practical importance for high-speed robot systems to prevent centripetal force from flipping over legged platforms or dumping unsecured loads off of differential or holonomic robots (e.g., material handling, delivery).

The Hybrid-A* planner provided by Nav2 within the *Smac Planner* framework provides a few additional unique variations. Rather than using solely binary collision information, all values in the cost grid are utilized to steer the feasible search via an admissible cost-aware obstacle heuristic function. This leads optimal paths into lower-costed regions away from obstacles, respects user generated cost grid constraints (such as keepout zones or penalizing narrow spaces), and improves path quality.

This is sufficiently effective that expensive post-processing of the paths proposed in [14] is not strictly necessary to improve paths' smoothness and safety margins from obstacles. Moreover, because expanded search nodes are generally farther from obstacles, optimizations in collision checking are made which further improve run-time performance.

Additionally, limits are placed on the maximum length of analytic expansions to prevent excessive bypassing of heuristic search. Limiting the expansion distance prevents suboptimal behaviors that would have been penalized in the search and/or cost grid, such as reversing for long distances or entering sensitive high-cost zones. The role of analytic expansions are to find a path to the exact goal, thus, it is most applicable in close proximity to the goal pose. Reducing the expansion length impacts performance negligibly. A typical run-time for this planner is between 10–250 ms in practical large deployed environments.

State Lattice was utilized by Carnegie Mellon University in the DARPA Urban Challenge [15,16]. Like Hybrid-A*, it uses continuous coordinates associated at discrete cells to create feasible paths from motion primitives. However, it leverages offline computed motion primitives that establish a state lattice graph rather than expanding using trees. It expands A* as a structured lattice graph and connects primitives at precise bin quantizations rather than at arbitrary locations. The minimum control set, or motion primitives for State Lattice, may be computed based on an arbitrary motion model, not only for Ackermann vehicles or restrictions based on minimum curvature. This can handle novel or atypical drive-train constraints, such as those found on planetary exploration rovers, legged robots; or differential-drive in-place rotations and omnidirectional lateral motions [17]. Thus, it is a generalized class of planner for any robot drive-train type when matched with a complimentary minimum control set – making it a powerful addition to Nav2.⁶

The State Lattice planner in Nav2 provides a number of noteworthy optimizations to improve performance beyond historical libraries such as SBPL, to 20–300 ms [18]. Classical State Lattice planners struggle to efficiently determine the best route through complex scenes using only distance-based heuristics. They also labor to find paths to precise goal poses with solely search. The cost-aware obstacle heuristic and analytic expansions are also shared with this planner to resolve these issues, respectively. In providing the *Smac Hybrid-A** capabilities, this planner improves performance by an order of magnitude and provides remarkably similar results to Hybrid-A* both in compute time and path length/quality, show in Table 1. The path lengths and planning times are within 0.5% and 1.6% of each other, respectively. This is a powerful result: Nav2 contains two planners with analogous behavior each covering important niches to establish support for the breadth of possible robot platform types. Therefore, effectively any robot base can be used with Nav2 to create kinematically feasible plans, which are also akin to any other robot type's plans – forming consistency across heterogeneous fleets. Fig. 2 illustrates two paths produced by Hybrid-A* and State Lattice planners.

Note that both of these planners and the 2D-A* discussed previously derive from the *Smac Planner* framework. A number of shared optimizations, pre-computations, and caching have been made within the framework to be competitive with established planning algorithm implementations in planning time, behavior and path length. The *Smac Hybrid-A** and State Lattice planners are notably faster than their 2D-A* counterpart in large part due to an initially reduced cost-aware obstacle heuristic resolution used to steer the feasible search, which is later upsampled as nodes are visited.

⁶ The provided control set generator yields curvature minimizing trajectories with optional in-place rotations and lateral motions to support Ackermann, Legged, Diff., Omni. robots.

Table 2
Trajectory planners comparison.

Local trajectory planner	Max. frequency (Hz)	Type	Circular	Non-Circ.	Diff.	Omni.	Ackermann	Legged
Model Pred. Path Integral ^a	125	Predictive	✓	✓	✓	✓	✓	✓
Timed Elastic Band	130	Predictive	✓	✓	✓	✓	✓	✓
DWB	250	Reactive	✓	✓	✓	✓	–	–
Graceful	1,800	Control-law	✓	✓	✓	✓ ^c	–	–
Reg. Pure Pursuit ^a	>4,000	Geometric	✓	✓	✓	✓	✓	✓
Rotation Shim ^{a,b}	>>10,000	Kinematic	✓	✓	✓	✓	–	–

An inter-modal algorithm selection summary can be found in [Appendix A](#).

^aExclusive to ROS 2.

^bTo be paired with other algorithms.

^cWhile not taking full advantage of the omni-directional base, this control law works well on powered caster bases which are not fully holonomic.

3. Overview of local trajectory planners

Once a global path planner or similar has found an acceptable route through the environment, another technology needs to convert the intended route to navigate into a stream of velocity commands for the mobile platform to track. These commands generally attempt to follow the path, but may also consider objectives such as reducing likelihood of collision, improving smoothness, minimizing important objectives (curvature, acceleration, jerk), or may implement other primitive behaviors. This is the work of the (Local) Trajectory Planner, which generates collision-free trajectories and issues current velocity commands to the base's motor controller. This completes the Hybrid Planning schema introduced in Section 2, in that global planners find an acceptable route and trajectory planners follow it, potentially considering additional system constraints.

Due to its close coupling to the mobile robot base, trajectory planners are often referred to as *controllers* and the selection of the best trajectory planner will be highly dependent on the robot kinematics. For an abbreviated set of recommendations see [Appendix A](#).

There are a wide variety of trajectory planning techniques to satisfy different robot base types and applications. The behavioral needs of an industrial warehouse automation robot differs as much from a hospitality robot as a circular differential-drive base differs from an Ackermann-steering car. Thus, there are different classes of techniques such as reactive, predictive, geometric, machine-learning, and control-law derived trajectory planners. Furthermore, trajectory planners often need specialized tuning to match the kinematic and dynamic characteristics of a particular robot base. While techniques are portable between robots, they typically require the most involved and intensive tuning of any system on a mobile robot.

Nav2 includes several implementations of trajectory planners to meet the spanning set of needs for differential-drive, omnidirectional, legged, and Ackermann style vehicles in both circular-optimized and arbitrary-shaped forms. Namely, the Dynamic Window Approach (DWA), Regulated Pure Pursuit (RPP), Model Predictive Path Integral (MPPI), and Rotation Shim controllers. In addition to the trajectory planners that are part of the Nav2 project, there are several trajectory planners available in the broader ROS 2 ecosystem, such as the Timed Elastic-Band (TEB) and the Graceful Controller. [Table 2](#) compares these controllers behaviors and key attributes.

Nav2 passes each trajectory planner a rolling-zone local cost map, planned path, and positional information to utilize along with any additional information it obtains itself through topics, parameters, or services. New to Nav2, run-time configurable plugin interfaces were added to abstract out commonalities between many controller implementations. These support dynamically set speed limits, navigation goal checking criteria, and progress checking algorithms.

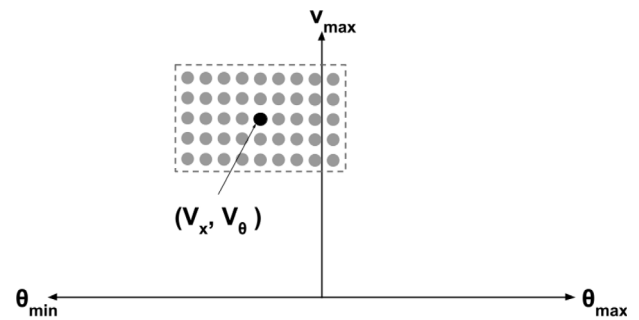


Fig. 3. Visualization of the dynamic window of feasible velocity commands from the robot's current state, with 9 angular velocities (v_θ) and 5 translational samples (v_x).

3.1. Reactive controllers

Reactive trajectory planners use knowledge of the surrounding environment to construct collision-free trajectories towards the objective. They do so separately from global planning to ensure obstacle avoidance which is *reactive* to changes in the local environment, which may be more fine grained, up-to-date, or information rich than that used in global planning. This knowledge may include sensory information in addition to map or occupancy information, though not definitionally. In Nav2, all planners use live sensor information fused with a map via its local cost map (discussed in Section 5).

Although reactive trajectory planners often consider the environment around it, they lack full context in temporal dimensions. These methods typically rely on potential fields or hand-engineered heuristics which score trajectories based on the robot's current state and the localized surroundings in the current time-step only. While this can be effective in a number of environments and boasts low computational complexity, it is often susceptible to local minima. Nonetheless, robust reactive control methods continue to find a great deal of application in modern robotic systems.

In particular, the **Dynamic Window Approach** endures as a method of interest to system designers and researchers alike. DWA is enabled by the DWB Local Planner in Nav2.⁷

The method sub-samples a set of feasible velocity commands from the *dynamic window* of achievable velocities, shown in [Fig. 3](#). These are a set of candidate commands from the robot's current velocity state (v_x, v_θ) within its dynamic limits [19]. This set contains constant velocities which create circular arcs when projected forward – a simple trajectory. Each arc is scored against a set of critic functions and the best weighted average velocity is

⁷ DWB as in the 'next' after DWA. It is the successor of the DWA controller found in the ROS (1) Navigation Stack.

selected for a planning iteration. The critic functions are mathematical operations which score the candidate trajectories based on various criteria optimizing for desired behavior (e.g. tracking the path and avoiding obstacles). The original objective function takes the form in Eq. (2):

$$\alpha \text{ heading}(v, \omega) + \beta \text{ dist}(v, \omega) + \gamma \text{ vel}(v, \omega) \quad (2)$$

where α , β , and γ are tunable penalty weights, $\text{heading}(v, \omega)$ is a measure of distance to the goal, $\text{dist}(v, \omega)$ is the distance to the nearest obstacle on the trajectory, and $\text{vel}(v, \omega)$ encourages moving at full speed.

Within the DWB package, this method is generalized and expanded. Rather than considering only forward translational motion, $\dot{x}$, lateral translation is also considered for omni-directional platforms, $\dot{y}$ (which may be set to zero for differential-drive platforms).

Further, dynamically configurable trajectory generation and critic functions may be used to expand or adapt behavior for a particular platform. This allows for customization of both the generation of candidate velocities and the scoring of those velocities. Currently the following critic functions have been made available for users: goal alignment, goal distance, path alignment, path distance, prefer forward motion, anti-twirling, anti-oscillation, and traversed cost by footprint or center values. Each contains its own relative weight and configurable parameterization to produce tunable behavior. In addition, two unique trajectory generators are also available. The *Standard Trajectory Generator* samples trajectories based on a configurable maximum time ahead to forward simulate, while the *Limited Acceleration Generator* finds trajectories based on a configurable simulation period, δt , to apply at each time-step.

DWB is currently the default local trajectory planner in Nav2, largely due to historical precedence of Base Local Planner and DWA Local Planner in ROS (1) [20]. DWB's architecture is a refactor of these controllers from the original Navigation Stack to be more flexible with plugin-based critic functions and trajectory generators. The configurability of DWB is unparalleled and has been shown to be capable of any number of reactive behaviors when configured and/or customized.

However, the method suffers from numerous, highly inter-related parameters that must be tuned to achieve such performance (e.g. choice of critic functions and their relative weightings). This process can be complex and poorly tuned DWB configurations can create extremely suboptimal behaviors. This tuning complexity has long been a source of critique for the ROS mobile robotics community. Many deploy un-tuned parameters and remark at poor robot behavior due to the trajectory planner, although the perception systems and global planning systems are quite mature. It is the maintainers' aim in the medium-term future to replace DWB as the default algorithm with MPPI — a well-tuned, modern predictive controller algorithm, discussed in Section 3.2, once it reaches an appropriate level of maturity.

3.2. Predictive controllers

Predictive trajectory planners are a specialization of reactive planners which consider temporal information in the process of constructing objective-maximizing trajectories. They refine the previous time-step's trajectory when evaluating a current request and typically model trajectories as non-trivial sequences of velocity commands at future time intervals. This *prediction* gives the classification its distinction and makes it more robust against local minima causing a system to become in an unrecoverable or 'stuck' state. Further, oscillation, sudden changes in velocity (and its derivatives), and other adverse behaviors are greatly reduced when using well-established methods — making a robot system

exhibit more intelligent behavior. Predictive controllers come at a cost however: they are far more computationally demanding than other approaches and are an area of evolving research.

Many predictive controllers are modeled and solved using optimization-based techniques under the umbrella of model predictive control (MPC). MPC methods have been shown to be effective on a broad range of problems within and outside of robotics, including autonomous driving and industrial applications. The two predictive controllers in Nav2 both belong to the family of MPC approaches, namely the Timed Elastic Band (TEB) and Model Predictive Path Integral (MPPI) controllers — of which both are unique to the ROS mobile robotics ecosystem.

Timed Elastic Band (TEB) is one of the most popular trajectory planners in ROS (1). Some global path planners generate plans that have frequent or abrupt changes in direction. Elastic bands transform such paths into smooth, deformable plans that are more appropriate for trajectory following [21]. In the original formulation, small subsets of collision-free space, known as "bubbles", are overlaid on the path and overlap in such a way as to guarantee collision-free movement between them. The path is then deformed by subjecting the bubbles to repulsive forces that represent distances from obstacles and contracting forces that eliminate excess slack in the path.

TEB is an extension of Elastic Bands that also accounts for temporal constraints on motion, such as acceleration and velocity limits [22]. The constraints are weighted and optimized using a numerical optimization framework.

The poses of the robot and time deltas between poses are optimized in TEB. Numerous constraints are considered in the problem formulation:

- *Waypoints* are attractive forces that encourage the robot to remain close to intermediate points along the original global path.
- *Obstacles* (both static and dynamic) represent repulsive forces that keep the path from colliding with objects in its environment.
- *Kinematic constraints* can be used to ensure that the optimized trajectory is kinematically feasible.
- *Velocity and acceleration* constraints ensure the final trajectory is dynamically feasible
- *Fastest path* constraints ensure that the optimized trajectory is temporally efficient.

In ROS 2, the TEB Local Planner package contains the TEB implementation [23]. It uses the g2o framework to carry out numerical optimization [24]. At each control iteration, the poses from some portion of the global path and estimated time deltas between them are added to the graph, along with the constraints described above. The graph is then optimized and the control command is extracted from the optimized TEB. To help constrain the problem, the planner uses the previous cycle's optimized trajectory as the estimated starting values for the optimization in the current cycle. Fig. 4 shows an example of an optimized TEB. The final trajectory accounts for the kinematic constraints of the differential-drive platform being used, while also adhering to the original global plan.

The package is highly configurable. As with other methods described, tuning these weighting parameters to produce optimal behavior for a given application can be time consuming. Furthermore, as with all numerical soft-constraint optimization techniques, no guarantees can be given on the optimized solution, e.g., kinematic constraints may not be perfectly respected, or an insufficient weight on the obstacle distance constraint may result in the TEB skirting close to obstacles. This is in contrast to sampling-based approaches like the Dynamic Window Approach,

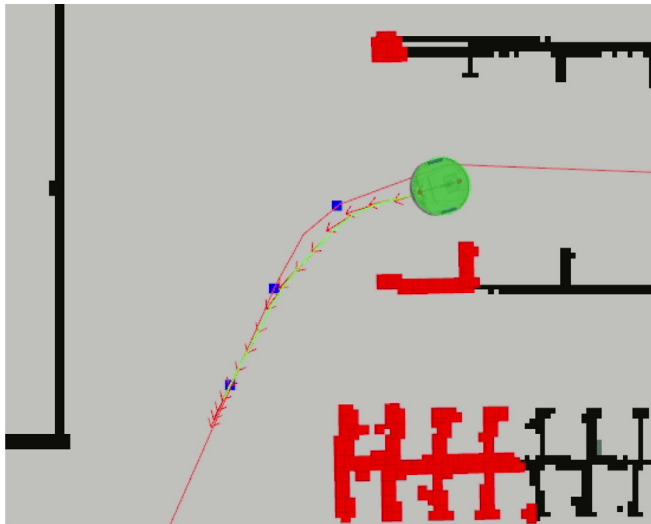


Fig. 4. Example of an optimized Timed Elastic Band. The robot's global plan is shown as a red path. Way points are shown as blue squares. The robot's cost map is transformed into a vector representation (red blocks) that is suitable for distance calculations in the optimizer. The optimized trajectory, shown in green, comprises poses that are depicted as red arrows. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

where kinematic constraints can be explicitly modeled during trajectory generation.

Model Predictive Path Integral is an MPC variant which uses perturbed trajectory samples to estimate the optimal trajectory [25]. Rather than numerically optimizing trajectories each time-step, a large number of noised samples are generated derived from the previous time-step's optimal trajectory, each is scored, and the optimal trajectory is identified. Since this method does not involve the non-linear optimization stage found in most MPC techniques, the cost functions are uniquely not required to be differentiable or convex — yielding greater latitude in designing system behavior. The iterative update law optimizes the trajectory between time-steps, refining it as the robot moves towards its objective. It has been proven effective in a number of domains including aggressive outdoor driving and commercial robotics.

In Nav2, the MPPI controller utilizes tensor representations to batch process trajectories. The trajectory planner generates many randomly noised controls from Gaussian distributions for each control axis (x , θ , and optionally y). The noised controls are added to the previous optimal trajectory's velocities to create a set of randomly perturbed trajectories. Constraints are applied on these trajectories' velocity limits and turning radii to make them physically achievable for omnidirectional, Ackermann, and differential-drive robots. Using time samples in the trajectory, the velocities are then projected to fully construct the trajectory poses.

Each trajectory is scored by a configurable set of critic functions — or objective functions which optimize for particular elements of robot behavior. Like DWB, all of the critic functions are dynamically loaded plugins and may be added, removed, or customized for a particular application. The following critics are provided and produce good high-speed path tracking and active collision avoidance behavior: goal angle, goal distance, obstacle map, path align, path angle, path follow, prefer forward, anti-twirling, and dynamic constraints. After scoring, the batch's costs are normalized and a softmax function is applied to return the optimal trajectory. The first control in the optimal trajectory is

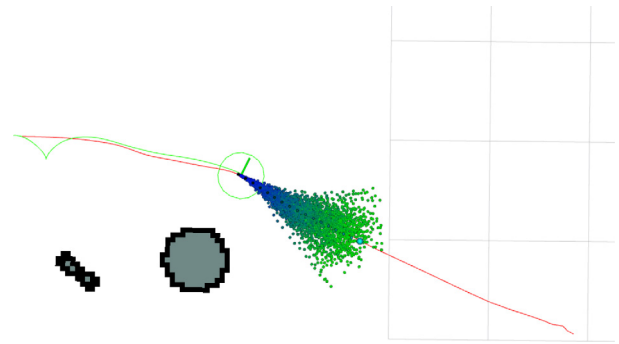


Fig. 5. Example MPPI controller in action; red is the original global path, green is the actual path traveled by the robot (circle). The candidate trajectories points are shown, with the optimal selected trajectory highlighted on top. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

then sent to the robot base controller to track. This process is conducted iteratively to refine the optimal trajectory each time. For the next time-step the trajectory is shifted, removing the first velocity which was sent to the robot base controller. This improves performance by starting at the next predicted location with the remainder of the as-of-yet not executed portion of the trajectory. Thus, it is important that this controller runs at a sufficiently high rate with a sufficiently large number of samples. Empirically, 1000 samples at 50 Hz or 2000 samples at 30 Hz yield good results and are easily achievable in this CPU-only implementation (Table 2).

MPPI behaviorally is a major improvement over reactive techniques. It is exceptionally effective at reacting to dynamic obstacles intelligently even when not explicitly modeled as dynamic in the cost map. Further, MPPI rarely requires active recovery behaviors to assist getting out of local minima due to becoming stuck or unable to make progress. Its predictive back-out maneuvers when in close proximity to static or dynamic obstacles is a considerable asset in high-trafficked or confined environments to prevent the robot from becoming stuck. Fig. 5 shows a similar behavior in isolation; whereas MPPI smoothly backs up and switches directions in a partial 3-point turn when a holonomic path begins in the opposite direction of the robot's initial heading. While there are performance and behavioral differences between TEB and MPPI, the major relative strengths of MPPI is in its software architecture and test coverage for long-term maintenance, simpler and more intuitive configurations, and active maintainer presence.

3.3. Geometric and control-law controllers

As opposed to reactive and predictive controllers, geometric and control-law derived controllers are quite simple. Their aim is to follow a given reference path or objective without deviation. Many of these techniques were early undertakings in trajectory planning, but have had an immense amount of staying power. While dynamic behaviors give a robotic system some of its most directly visible intelligence, that is not necessarily optimal in some practical applications. For example, exact route following behaviors have utility when following pre-planned routes through approved corridors in industrial settings where safety is paramount. Thus, it is an important type of behavior to support in a complete navigation framework.

These controllers do attempt to prevent collisions utilizing the cost map information given. However, rather than adaptively avoiding collisions, these controllers will simply stop without going around an obstacle in its way. They will wait for a new global route to be selected or the obstacle moves from its

path. Due to their simplicity in evaluating geometric, kinematic, and/or control-law derived functions, these controllers can run well in-excess of 1 kHz.

Regulated Pure Pursuit is a variation on the traditional Pure Pursuit algorithm. Pure Pursuit finds a point on the reference path a fixed-distance away from the robot to track, known as the *lookahead point* [26]. This fixed-distance from the robot can be interpreted as the chord of a circle with points on it $(x, y)_{vehicle}$ and $(x, y)_{lookahead}$ belonging to the positions of interest. When the path is represented in the vehicle coordinate frame, the curvature of the route, κ , to follow the lookahead point can be represented as $\kappa = \frac{2y}{L^2}$, where y is the lateral coordinate of the lookahead point and L is the distance away from the robot. With an arbitrary choice of translational velocity v_t , we can compute an angular velocity, v_a , required to drive the robot towards the lookahead point via $v_a = v_t \kappa$. This process is conducted iteratively resulting in the robot tracking a given reference path.

Adaptive Pure Pursuit builds on this by dynamically adjusting the lookahead distance based on the robot's current velocity, typically in the form $L = v_t l_t$, where l_t is the lookahead time gain [27]. This process has been shown to improve tracking stability of reference paths and reduce tracking errors in high curvature turns.

The Regulated Pure Pursuit controller builds on these foundations and expands the behavior one step further with heuristic penalty functions on the desired translational velocity v_t to regulate the speed of the robot in confined or precarious situations [28]. Rather than naively assuming a fixed speed, which can be dangerous in practical applications, the robot is slowed when close to obstacles or the curvature of the turn exceeds configurable limits. This slows the robot before sharp blind turns into partially observable settings (e.g. into or out of hallways and aisles) and reduces speed when close to other objects where the risk of collision is particularly high.

This has been shown in experiments to decrease stopping distances for emergency stops, reduce overshoot and undershoot in high-curvature turns, all without substantially increasing time-to-goal metrics. The minimum velocity of the two heuristics is used, Eq. (3):

$$v_{curv} = \frac{v_t}{r_{min} \kappa} \quad v_{prox} = \begin{cases} v_t \frac{\alpha d_O}{d_{prox}} & d_O \leq d_{prox} \\ v_t & d_O > d_{prox} \end{cases} \quad (3)$$

where r_{min} is the minimum turning radius to apply the heuristic curvature heuristic, d_O is the distance to the nearest obstacle, d_{prox} is the proximity to obstacles to apply the curvature heuristic and α is a tunable weight. Additionally, direction changes (e.g. forward to reverse) computed by feasible planners are used in RPP to switch directions as necessary at pre-planned poses.

Graceful Controller aims to provide smooth path following and automatic speed regulation for differential drive bases [29]. This trajectory planner is best used with a kinematically feasible global planner.

This controller is based on a geometric control law derived for differential drive robot bases which generates only feasible commands [30]. The control implements a fast and slow subsystem. The slow subsystem is implemented as an Archimedean spiral that will take the robot from the current pose to a target lookahead pose, based on a single parameter, $k1$. The fast control subsystem then determines the linear and angular velocity required to follow the spiral based on the $k2$ parameter. The default values for $k1$ and $k2$ suggested by original paper appear to work well on a variety of platforms. The translational speed is automatically reduced from the maximum allowable speed when the curvature of the path tightens based on two parameters, $lambda$ and $beta$. While [30] contains the complete equations involved,

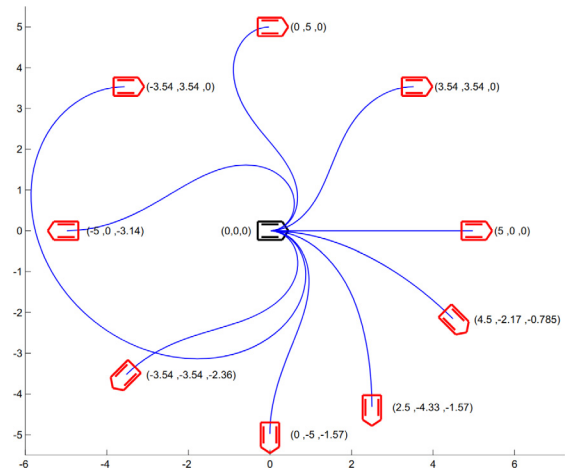


Fig. 6. Graceful Controller's behavior achieving non-trivial local poses [30].

higher values of $beta$ cause the overall drop in velocity to increase, while higher values of $lambda$ cause the rate of velocity reduction to be increased.

As with the RPP controller, the control law works on a single target pose (e.g. lookahead point). The controller works backwards through the global plan looking for the farthest point that is less than a maximum distance away from the robot and for which forward simulation of the control law to convergence results in no collisions. Selecting a pose too close to the robot can cause instability in the control law, so a minimum distance may also be defined. When no target pose can be found, the global planner is called upon to deliver a new path.

Unlike the RPP controller, the Graceful controller can produce smooth trajectories to non-trivial target poses without intermediate paths to follow due to its control-law based on spirals, Fig. 6. RPP and all Pure Pursuit variants are simple geometric approaches that requires continual updating of goal poses to produce path following behavior; for any single goal pose, computing a single v and ω to drive towards it.

Thus, the Graceful controller is a relatively more intelligent path tracker that can accomplish a variety of other tasks (e.g. docking or object following) – however RPP still has great utility in exact path following behaviors, including respecting direction changes in feasible plans.

A notable mode of operation is preferring final rotation. When enabled, the orientation of the final pose in the global plan will be replaced by an orientation that is tangent to the path. The robot will follow the path to the final pose and make a final in-place rotation to the desired heading. This behavior avoids a possibly large divergence from the planned path as well as large arcing behaviors.

Other features implemented in the controller include an orientation filter to clean up global plans and optional inflation of the robot footprint at higher speeds to increase safety margins (having a similar effect to RPP slowing when close to obstacles).

Rotation Shim is a re-usable component for robot platforms that may rotate in-place to rotate to a path's relative heading before beginning to track. This handles a frequent use case where controllers need to rotate a robot towards a newly planned path starting at another relative heading (e.g. from a holonomic planner from Section 2.1). Once the robot's heading is within a configurable angular distance from the path's initial heading, the primary controller algorithm is used to track the path. Controllers like DWB and MPPI are known to spiral out in a large arc towards the new heading or perform partial 3-point turns, respectively,

Table 3
Path smoothers comparison.

	Time (ms)	Length (cm)	Ave. cost	Max cost	Smoothness	Ave. turning radius (m)
Unsmoothed path	–	1150.47	12.26	129.46	76.45	0.71
Simple Smoother	0.90	1115.39	18.89	132.89	58.79	3.07
Constrained Smoother	20.82	1149.37	6.88	103.98	87.85	2.59
Savitzky–Golay Smoother	0.06	1146.75	12.18	127.76	76.19	2.29

An inter-modal algorithm selection summary can be found in [Appendix A](#).

when requested to track a path at a substantially different relative heading to its starting pose. This enables high-quality in-place rotations and removes the need for trajectory planners to process this special, but common and sometimes difficult initial task.

4. Overview of path smoothing

New in Nav2 are smoothing capabilities to refine global paths. Smoothers have long been employed in sampling-based planning algorithms to interpolate between points and smooth out irregular movements. While plans produced by search-based planners are typically less tortuous, many can be improved with additional refinements: 2D-A* can smooth jagged motions due to Moore search neighborhoods, Theta* can obtain continuous transitions where line segments meet, and feasible planners can benefit from curvature minimization.

Canonically, local trajectory planners have been successful in producing relatively smooth behavior from unprocessed global paths while considering dynamics and other constraints. However, improving path quality can further reduce jitter in geometric and control law-based path tracking, as well as improve stability for predictive and reactive algorithms [31]. Thus, Nav2 provides three path smoothing algorithms to support holonomic and feasible planners – the Simple Smoother, Constrained Smoother, and Savitzky–Golay Smoother.

Table 3 show a comparison of the path smoothers, where the costs reported are those in the cost map and smoothness is measured as $|\Delta x_{i+1} - \Delta x_i|$, ($\Delta x_i = x_i - x_{i-1}$). These are the results of an experiment composed of 1000 randomly generated start and goal poses in a representative cluttered home environment (e.g. furniture, hallways, open rooms, random objects strewn about) using the Smac Hybrid-A* planner. These paths were then smoothed using the algorithms in this section and compared on a set of key metrics: compute time, path length, cost or distance from the nearest obstacle, smoothness, and path curvature.

Simple Smoother is used to remove localized imperfections such as oscillation, discontinuities, and turning transitions in the holonomic planners, typically in 1–6 ms. As its purpose is to remove localized irregularities or discontinuities, it uses a simple gradient descent formulation which minimizes the objective function:

$$\sum_{i=1}^{N-1} \omega_s (\Delta x_{i+1} - \Delta x_i)^2 + \omega_d (x_i - y_i)^2 \quad (4)$$

where ω are configurable weights, x is the smoothed path and y is the original path. The first term is commonly used and optimizes for smoothness, while the second term penalizes adjusting the smoothed path too far from the initial path. Together, they provide smoothed behavior in the local neighborhood of the unsmoothed path resolving defects such as sharp turns, jagged regions, and odd characteristics produced by gradient descent on discrete grid maps. This is born out in the experimental results – this smoother is quite fast and generates improved paths. However, because this will smooth out turning corners, it may increase the path costs relative to the initial plan as the smoothed result becomes closer to obstacles (ex. [Fig. 7](#)).

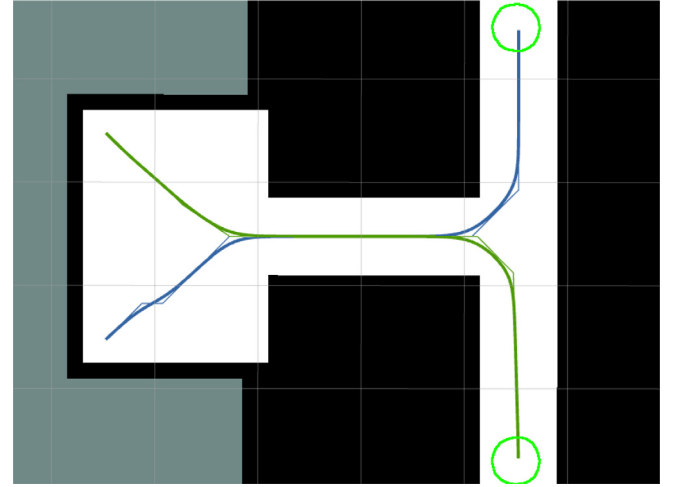


Fig. 7. Theta* and Smac 2D-A* paths smoothed by Simple Smoother. Theta* depicted in green, Smac 2D-A* – in blue. Non-smoothed paths depicted in narrow lines, smoothed – in bold. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

The newly optimized path has its via-point orientations assigned as the tangent to the path. The smoothing terminates when the parameter tolerance is achieved or the maximum time has been used. If a path is found to be in collision, the last collision-free path from prior iterations is returned. [Fig. 7](#) illustrates the Simple Smoother for raw paths produced by Theta* and Smac 2D-A*.

Constrained Smoother is designed for refining the feasible planners from 2.2 with a more developed optimization cost function taking into account curvature, cost fields, and smoothness. It derives from the smoothing algorithm introduced in Hybrid-A*, but rather than using a Voronoi Field, it leverages the existing cost grid with obstacle inflation [14]:

$$\begin{aligned} & \omega_s \sum_{i=1}^{N-1} (\Delta x_{i+1} - \Delta x_i)^2 + \omega_c \sum_{i=1}^{N-1} (\text{cost}(x_i, y_i))^2 \\ & + \omega_k \sum_{i=1}^{N-1} \sigma_k \left(\frac{\Delta \Phi_i}{|\Delta x_i|} - \kappa_{\max} \right) \end{aligned} \quad (5)$$

where ω_s , ω_c , and ω_k are the weights for the smoothness, cost, and curvature terms, respectively; σ_k is a quadratic penalty function, $\kappa_{\max}$ is the maximum feasible curvature, $\Delta \Phi_i$ is the change in tangential angle, and $\text{cost}(x_i, y_i)$ is the cost at a given cell. This objective function is solved using Google Ceres, providing a number of linear solver options.

This provides more comfortable paths for feasible planners by minimizing curvatures (from their maximums in search) to improve practical drivability while retaining kinematic feasibility. It also balances improving the smoothness of the path with shifting the plan into lower-cost regions. Since it has no constraint on the original path coordinates, this smoother may make larger, global level changes to the path beyond localized artifacts. Thus,

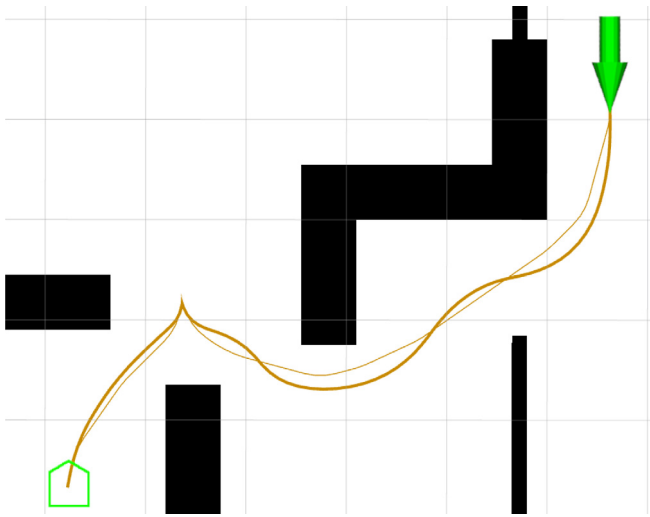


Fig. 8. Smac Hybrid A* path smoothed by Constrained Smoother. Non-smoothed path depicted in narrow line, smoothed — in bold.

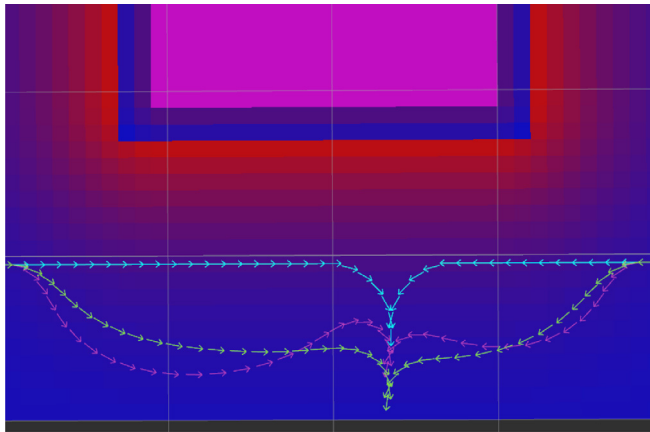


Fig. 9. Impact of increased cusp costs near direction-changing points. The cyan path is the unsmoothed original path, purple is using the same ω_c , and green uses ω_{cc} near the direction change. The cost-field is shown as the background colored gradient near a rectangular obstacle [32]. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

the benchmark shows that the Constrained Smoother generated paths typically longer than the Simple Smoother, but with significantly reduced costs, while also improving on smoothness and curvature from the original path. Fig. 8 demonstrates this visually with a path of similar length, but maximizing distance from obstacles and reducing path curvature.

When reversing path segments exist, ω_{cc} (cusp-cost) is used instead of ω_c within close proximity to cusp points. This value is strictly larger than ω_c and incentivizes the optimization problem to push direction-changing maneuvers further away from obstacles for additional safety margins. Fig. 9 shows an example of the impact of an elevated ω_{cc} in close proximity to cusp points.

While the Constrained Smoother is targeting feasible planners, it may also be used by the holonomic planners. The curvature term is removed when $\omega_c = 0$ and may operate on paths using only the cost and smoothness terms to perform global-level optimization rather than only removing localized imperfections and discontinuities. However, this smoother is much more computationally expensive than the Simple Smoother, taking up to several seconds depending on the length of the global path and the

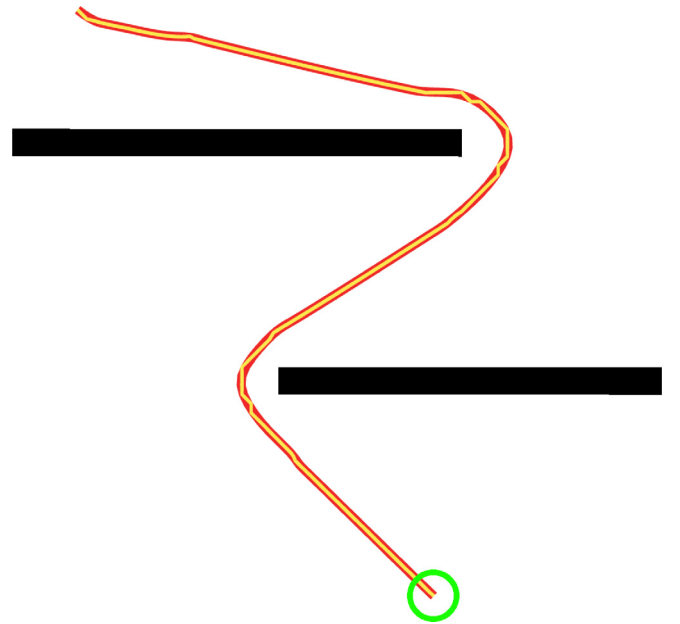


Fig. 10. NavFn path (yellow) smoothed by Savitzky–Golay Smoother (red). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

environment. Thus, it is primarily recommended where maintaining feasibility is a strict requirement. The smoother terminates when the parameter, function, or gradient tolerance is achieved, the maximum number of iterations has been reached, or the maximum time has been used.

Savitzky–Golay Smoother is a path smoother derived from the famed digital filter of its namesake [33]. This filter analytically smooths regularly-spaced data using a sliding window polynomial function to remove noise from the signal. This may be applied to path plans from search-based planners due to their regular search intervals. This smoother is unique from the others in that it will not generally modify the character of the input path. Rather, it will only smooth out irregularities that can be modeled as noise within a 7-point window. This is most useful for gradient-descent path creation, as found in the NavFn Planner or potential field methods, which can generate irregularities due to performing gradient-descent on a discrete grid. This method will smooth “noisy” outlier path points while retaining the intent from the planner. Fig. 10 shows a case in which NavFn generated a path with an unusually large numbers of jagged points, which the Savitzky–Golay Smoother refines into a smooth global path. Since the character of the path remains the same, the changes in key metrics are often negligible. This smoother, like the others, will also update the orientation vectors of each path point to be tangent to the final smoothed path.

5. Overview of perception

5.1. Overview

The key to achieving reactive behavior – where a robot intelligently navigates in a dynamic environment – is using the robot’s sensors and semantic information to maintain a representation of the environment. ROS 2 employs *cost* or *risk maps* as the environmental model to consolidate the results of potentially many algorithms. A balance must be achieved to create a sufficiently high-fidelity representation of the environment and be able to maintain that representation efficiently. Perception algorithms,

sensor data, or semantic information about the environment are populated into this world model for use in global path and local trajectory planning.

Cost maps reduce the world model into a two-dimensional grid, with each grid cell having a cost associated with it. Cost maps evolved from the probability-based occupancy grids developed at Carnegie Mellon in the 1980s [34,35]. The modern cost map was first described by Marder-Eppstein et al. [3], where the cost associated with each cell was an integer in the range [0, 255]. In general, planning algorithms will prefer paths with lower costs and avoid those with higher costs, but there are also special values for unknown and occupied cells (255 and 254, respectively) to provide hard constraints on robot behavior. These special values allow planning and collision checking algorithms to guarantee viable navigation in only free and known regions. The cost map acts as the planning algorithms' configuration space [36]. Fig. 11 shows an example of the key values in a cost map.

The cost map provides a reasonably balanced approach to the fidelity/efficiency trade-off on both low and high power compute platforms. The grid data structure provides a simple correspondence with the real world and also provides constant time lookup. One drawback is that the memory consumption required to maintain the cost map scales linearly with the number of cells. The use of Quad-trees to represent occupancy grids has been proposed to alleviate this scaling via multi-resolution data storage [37]. However, these methods fail to recover substantial memory benefits when cost map convolution or inflation are applied, or when the density of obstacles in the environment leaves little open space to represent with coarser-resolution quantizations – while also introducing non-constant look-up times. While these are useful developments for some applications, it is most common for users in ROS 2 to apply a form of cost map inflation and operate in restricted spaces. In practice, the size of the cells in the cost map is 0.05 m by 0.05 m, which typically is small enough to render the environment with sufficient fidelity, but also large enough to not require enormous amounts of computation.

While a two-dimensional cost grid discards a great deal of potentially useful information, it has been applied repeatedly to solve a great number of practical robotics challenges. Further, it is common for perception algorithms populating the cost map to maintain higher fidelity representations internally for modeling the environment, which are binned or reduced to two dimensions for consolidation with other algorithms. Maintaining a two-dimensional cost map enables the application of a broad number of highly-efficient planning, control, and collision checking techniques for arbitrary types of robot systems.

The process by which the cost map is updated and maintained in ROS 2 is the *Layered Costmap* [38]: an extensible, systematic way to update the cost map with many, configurable, and arbitrary perception algorithm results. The Layered Costmap comprises an ordered list of dynamically run-time loaded layers which each represent a different data source, algorithm, or result. Each update cycle, the main cost map is updated by polling each of these layers in turn to populate the grid with new information.

Any developer may write a costmap layer and include it on their robot system for their specific applications. Common types of layers include: cost map inflation, sensor processing, semantic information, results from machine learning detection or segmentation algorithms, and multi-robot coordination. However, a set of common layers are provided by ROS 2 applicable for a broad range of common applications.

5.2. Costmap layers

Static A map provides information about where obstacles are known to be a priori, often from an offline mapping algorithm or



Fig. 11. Cost map example in a practical home environment. Black are lethal obstacles, light blue are inscribed lethal costs due to robot dimensions, the gradient from red to blue are a cost potential field caused by the inflation layer, and gray are unknown cost cells. Free space with no potential cost are white, not shown. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

other data source like blueprints. The static layer subscribes to an *OccupancyGrid* topic containing this information. It is common for the static layer to be the first costmap layer, providing the base information about the environment for later algorithms to modify. While the name implies non-changing, the static layer can have the map change over time as the result of map sharding over large spaces or updates to the environment due to continual mapping. There is a separate subscription to a *OccupancyGridUpdate* topic for efficient non-dimension changing updates to localized map information.

Obstacle The obstacle layer stores information from high accuracy sensors such as lasers and RGB-D cameras in a two-dimensional grid. Each point in the sensor data is treated as a ray, originating from the sensor pose. That ray is ray-traced through the grid in two dimensions using Bresenham's algorithm, where the endpoint is marked as occupied and its path is marked as free space due to direct visibility. Fig. 12 demonstrates this process visually. The 2D laser scanner produces measurements from obstacles which are marked in the grid in blue. Bresenham's ray-casting algorithm is used to clear out the visible free-space from the sensor to the obstacle in white, while the remainder of the grid is still unknown (gray). Due to the high accuracy of the sensors, these observations are treated as absolute, without the probabilistic calculations of occupancy seen in the 1980s. The simplification of logic also assists in quicker computation. As a robot moves through the scene and/or more measurements are taken, it will dynamically generate an occupancy map of the environment around it based on depth sensing.

The two dimensional nature of this layer makes it ideal for planar laser scanners, but may be overly constrained when using three-dimensional data. It may be necessary when working with low-accuracy or noisy sensors to pre-filter the data stream before processing in the obstacle layer to reduce noisy results in the cost map.

Voxel The voxel layer is similar to the obstacle layer, but tracks the environment and ray-traces measurements in three dimensions. It is most useful for RGB-D, 3D laser scanners, or other non-trivially two-dimensional sensor streams. Internally, it uses a voxel grid model the size of the cost map to represent the environment in a non-probabilistic way. This model scales linearly with cost map size, similar to the two-dimensional grid, by maintaining the height dimension through manipulating the bits of an unsigned 32 bit integer. Thus, its maximum height is limited. It is recommended to only track heights to the limit of

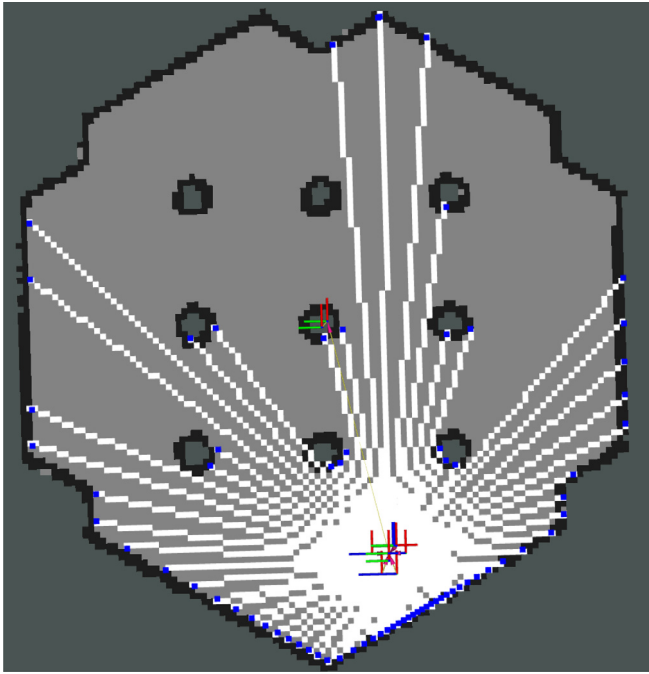


Fig. 12. Visual demonstration of ray-tracing based marking and clearing in a grid map. Overlaid on a sandbox map, a low-resolution laser scan is raytracing freespace (white) from measurements (blue). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

what is useful (e.g., a home vacuum robot need not maintain a voxel grid > 1 m off the ground). The vertical resolution of the voxels is independent from their horizontal resolution, and is typically larger. The three-dimensional data is projected down to two dimensions for writing into the main cost map. As it conducts three-dimensional raytracing, this layer is more computationally expensive than the obstacle layer.

Non-Persistent Voxel The non-persistent voxel layer computes values in the same manner as the voxel layer, but does *not* persist the data from one update to the next. Thus, each iteration a new voxel grid is created to populate the cost map, then cleared.

Spatio-temporal Voxel The Spatio-temporal voxel layer (STVL) use a more refined data structure than the voxel layer, Fig. 13. It tracks obstacles in three-dimensions using OpenVDB, a sparse voxel grid library developed by Dreamworks Animation for movies such as *How to Train Your Dragon* and *Puss in Boots* [39]. Rather than ray-tracing to clear obstacles in the voxel grid, STVL uses a method called *decay acceleration* to maintain a representation of the environment based on the sensor's measurement frustum and current measurements. As measurements are processed, the timestamp of the data is added to the voxel belonging to the reading. If an active (e.g. occupied) voxel is within the sensor frustum and seen by new data, the time is updated. If a voxel is not viewed by new data, a decay acceleration factor is applied to its time, quickening its removal due to lack of current visibility. The frustum models are configurable and currently support 3D lidar 'donut' models and RGB-D, depth, and similar sensors using a 6-plane bounding volume defined by the minimum and maximum range and horizontal and vertical field of views.

For all voxels outside of the sensor's frustum, a global linear time decay is applied, usually 10–30 s. After voxels are expired due to either method, they are removed from the data structure. Thus, its storage size is only dependent on the number of currently active and occupied cells. The sparse voxel data structure

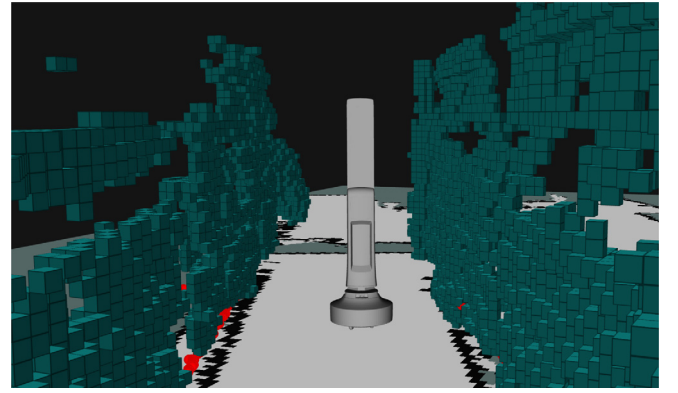


Fig. 13. STVL's internal sparse 3D voxel grid representing shelves in a retail environment [40].

is projected to two dimensions for inclusion in the main cost map. This layer is most appropriate for robots operating in highly dynamic environments with a high degree of sensor coverage. It was created to resolve issues with discrete cell ray-tracing which occasionally left uncleared obstacle cells in highly dynamic environments with many moving agents.

Range The range layer processes data from Infrared, Sonar, and Ultrasonic sensors. Due to the noise inherent to these types of sensors, the range layer tracks occupancy using a probabilistic model in which only cells with high confidence predictions are written into the main cost map. It has support for both fixed reading (e.g. binary detection of obstacles) or variable reading sensors. The probability of an obstacle in a cell is calculated using the static properties of the sensor (range of view, maximum reliable range and sensor variance), the sensor reading itself, and the angle and distance of the cell relative to the sensor origin [41].

Inflation The inflation layer uses the occupancy information from the other layers to add an inflated exponential decay function around untraversable regions. It does this by iteratively expanding from known obstacles to a configurable distance, applying Eq. (6):

$$\text{cost}(x, y) = (\text{cost}_{\text{lethal}} - 1) e^{-\omega_{\text{scale}}(d_o - r)} \quad (6)$$

where ω_{scale} is the cost scaling factor, d_o is the distance to the obstacle, $\text{cost}_{\text{lethal}}$ is the lethal obstacle cost, and r is the inscribed radius of the footprint. For circular robots, the cost map regions that are traversable after this is applied represent the configuration space of the robot. For non-circular robots, the cell cost for the largest distance between an edge of the robot and the center can be used with Eq. (6) to recover the cost at the center where full footprint collision checking is required. When the center cost is below that value, it is known that the robot cannot be in collision. Thus when using non-circular robots, it is important to still use inflation to optimize collision checking used in such algorithms as found in the Smac Planner framework described in 2.2. It is recommended to set a smooth inflation decay across the cost map, rather than maintaining only steep costs near obstacles, as shown in Fig. 11.

Keypoint The keypoint layer (a.k.a. keypoint filter) uses input masks to mark cells as obstacles. These masks are highly configurable to contain a great deal of semantic information and interpreted through `CostmapFilterInfo` into cost values to apply to the grid. These need not be only binary obstacle information, but also setting a range of costs to weight certain regions over others.

Speed The speed limiting layer uses the same filters API as the keypoint zones, but embedding maximum speeds as either

percentages or absolute values in the filter masks. This layer allows users to specify zones at which a robot should slow down, such as when entering or exiting aisles.

Binary The binary layer uses the same filters API as previously introduced, but embeds binary behavioral triggers into a spatial mask. When the cost value crosses the threshold, a ROS message is sent to a client server to indicate a change in binary state. An illustrative example would be to set spatial bounds conveying locations where it is and is not appropriate to stream camera feeds to a robot operations center. There may be regions of homes (such as bathrooms) and other facilities where remote monitoring of camera data would breach legal or ethical bounds. This layer allows the native embedding of spatial constraints to trigger binary actions.

6. Overview of behavior trees

In this section, we describe the architecture of how robotic components previously described are assembled to create autonomous navigation. Configurable behavior trees (BTs) are used to orchestrate internal robot navigation tasks such as planning, control, high-level behaviors, recoveries, and more in Nav2 [2,42]. This contrasts historical uses of state machines and hard-coded logic for designing navigation systems: typically passing a path to a path tracker with hand programmed behaviors [3]. While it is possible to model most BT configurations with hierarchical state machines (and vice versa), behavior trees have been the subject of much interest and are increasingly preferred from the results of user studies. The *BT Navigator* is the highest level component of Nav2, taking requests and processing them via the behavior outlined in the tree. Each behavior tree is modeled as an XML file comprising many BT nodes which are dynamically loaded at run-time. The behavior tree can be specified per-run so a system may take on different behaviors or tasks.

Most BT nodes will invoke a remote server over a ROS 2 interface (action, topic, service) to compute a value for component-level modularity, although this is not required. A growing number of primitive BT nodes are provided by Nav2, shown in Table 4, that may be used to develop useful robot behaviors. Note that users may easily create and use their own BT nodes and trees.

Nav2 provides a number of behavior trees comprising these nodes. These fall into three major categories: navigate to pose, navigate through poses, and task-specific applications. Each have several customizations available to provide intelligent behavior in various situations.

The navigate-to-pose behavior trees provide basic pose-to-pose navigation. The BTs have support for consistent replanning at a fixed frequency,⁸ on distance traversed,⁹ proportional to robot speed,¹⁰ or when the goal is updated.¹¹ However, it is sometimes desirable for robotic applications to not dynamically replan (except when strictly required due to a potential future collision) to produce predictable behavior.¹²

Practical BTs typically include recovery behaviors to resolve the vast majority of errors that may occur during navigation due to dynamic scenes, uncertainty, or confined environments. ROS 2 includes BTs that invoke context-specific actions in case of particular component failures. They also contain a global recovery branch for handling system-level failures.¹³ More advanced BTs

Table 4

Included behavior tree nodes.

Behavior tree node	Description
Action	
Navigate to pose	Navigate from A to B
Navigate through poses	Navigate from A-Z through B...Y
Compute path to pose ^b	Plan from A to B
Compute path through poses ^b	Plan from A-Z through B...Y
Follow path ^{a,b}	Follow path
Clear costmap	Clear environmental model
Back up ^a	Reverse
Spin ^a	Rotate
Drive on heading ^a	Drive forward
Smooth path	Smooth path
Remove passed goals	Remove goals passed B...Y
Truncate path	Truncate path near goal
Wait ^a	Pause
Reinitialize localization	Reset localization
Assisted Teleop ^a	Tele-op with collision avoidance
Condition	
Distance traveled	If a distance has been traversed
Goal reached	If the goal is reached
Goal updated	If the goal has changed
Initial pose received	If initial pose is set
Is battery low	If battery too low to continue
Is path valid	If path is not in collision
Is stuck	If not making progress
Is transformation available	If transform is possible
Is time expired	If time has passed
Path expiring timer	Timer before last path is stale
Decorator	
Rate controller	Control flow by set frequency
Distance controller	Control flow if distance has passed
Speed controller	Control flow proportional to speed
Single trigger	Trigger control flow once
Path longer on approach	If path too long on goal approach
Goal updated	Control flow if new goal set
Goal updater	Bypass and update goal over topic
Control	
Pipeline sequence	Flow-of-water sequence
Recovery	Executes second node if first fails
Round robin	Ticks children in round-robin style

^aIncludes complementary cancel task node.

^bIncludes complementary algorithm selector node.

include mixed-replanning¹⁴ and goal patience¹⁵ to resolve an assortment of commonplace behavioral issues, such as oscillation due to frequent replanning and handling temporary blockages, respectively.

The navigate-through-poses configurations provide a similar behavior but contain hard pose constraints for via-points to navigate through on the way to a final goal. These behavior trees make use of the 'Compute Path Through Poses' planning node rather than 'Compute Path To Pose'. In addition, this class of BT removes waypoints passed so that subsequent replanning iterations contains only future looking constraints.¹⁶ These configurations may also contain different replanning mechanics, recoveries, and advanced features as described prior.

The BT tasks are not required to be represented by a goal or via-points — or even full autonomy. Instead, a BT may be only interested in following a particular object¹⁷ with a collision free path or exploring new areas. Other examples include

⁸ navigate_to_pose_w_replanning_time.xml.

⁹ navigate_w_replanning_distance.xml.

¹⁰ navigate_w_replanning_speed.xml.

¹¹ navigate_w_replanning_only_if_goal_is_updated.xml.

¹² navigate_w_replanning_only_if_path_becomes_invalid.xml.

¹³ navigate_to_pose_w_replanning_and_recovery.xml.

¹⁴ nav_to_pose_with_consistent_replanning_and_if_path_becomes_invalid.xml.

¹⁵ navigate_to_pose_w_replanning_goal_patience_and_recovery.xml.

¹⁶ navigate_through_poses_w_replanning_and_recovery.xml.

¹⁷ follow_point.xml.

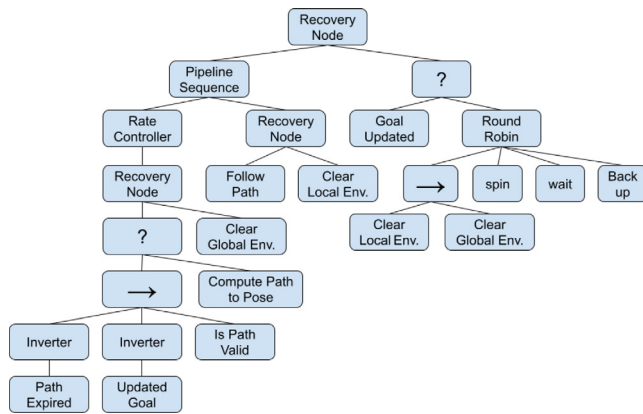


Fig. 14. Mixed-replanning navigate-to-pose behavior tree 14.

algorithm assisted remote tele-operation, odometry calibration experiments,¹⁸ and complete-coverage tasks.

As an illustrated example, Fig. 14 shows the mixed-replanning navigate-to-pose behavior tree with recoveries described in footnote 14. The left branch from the root node contains its navigation logic, while the right is used for system-level recovery. If a new goal is received during system-level recoveries, it will abort and reattempt navigation. There are also context-specific recoveries clearing the global and local environmental models within the navigation branch when path planning or local trajectory planning fail, respectively. In the navigation branch, mixed replanning is set to compute a new path if any of the following conditions are met: the path has expired after its duration of use, a new goal has been requested, or the path is found to be invalid due to sufficiently increased costs or collision. Thus, this BT will immediately replan if the path or goal requires it, but also refine periodically (typically 5–30 s). This enables responsiveness to urgent needs while providing refinement as new information is processed. Dynamic replanning is under a rate controller to limit the replanning rate slower than the tree's tick rate (100 Hz).

While not included in Fig. 14, it is advisable to leverage algorithm selector nodes as a recovery action to recompute a key function (path planning, trajectory planning) with another implementation. When backup implementations and contextual recovery actions are well selected for an application, they can resolve most issues. Further, the algorithm selectors allow a system to utilize many algorithms in their most useful contexts, rather than only a single method.

When designing behavior trees, it is prudent to create many independent nodes which are assembled in the tree's configuration for easy reusability and/or later customization. It is also wise to place as much logic in the tree as possible, in contrast to client servers, to provide the maximum flexibility. These principles were followed in the BTs outlined in this section, which has allowed them all to be composed from the same set of BT nodes from Table 4.

7. Overview of state estimation

For a mobile robot to successfully navigate through its environment, it needs to maintain an estimate of its state (pose, velocity, etc.). Such estimates are typically obtained through *sensor fusion*, or the process of determining a robot's state relative to a reference frame given many noisy sensor measurements, such as IMUs, wheel encoders, pose measurements, GPS, and more.

The output of this fusion is an approximation of a robot's state variables and typically a measure of confidence regarding the estimate.

ROS Enhancement Proposal (REP) 105 establishes the coordinate frames for common types of robotic systems to ensure interoperability of components [43]. The transformation tree containing these coordinate frames is populated by the algorithms of this and the next sections and managed by the *tf* library [44]. *tf* is utilized by a litany of applications to transform sensor data or relative vectors through time-varying reference frames — not simply for robot state tracking.

Within this formulation, the odom to base_link transform can be generated by integrating a robot's wheel encoder data to provide a pose estimate that can be used for local control of the robot. That pose will drift over time, however, so REP 105 also specifies a map to base_link transform¹⁹ that represents a drift-free estimate of the robot's pose. This pose is useful for carrying out operations such as global planning. However, it is subject to discontinuities, making it a poor choice for control algorithms. The map to base_link transform can be provided through a localization package, described in more detail in Section 8.

Alternatively, both transforms can be provided through the use of state estimators. In the case of the odom to base_link transform, this would typically mean fusing wheel encoder data with other continuous sources like angular velocity data from an IMU. For the map to base_link transform, the pose estimate from the localization package might be fused with other global pose estimates or with the same sources fused in the odom to base_link state estimator. This is useful if the localization package produces infrequent or irregularly-timed pose measurements, as it enables the global pose estimate to be generated at a fixed frequency.

While there are a myriad of algorithms for performing sensor fusion, two well-known and robust classes of approaches are Kalman filtering and factor graphs, both of which have implementations within the ROS 2 ecosystem.

7.1. State estimators

Robot Localization In the 60 years since its introduction, Kalman filtering has become one of the best-known and widely adopted approaches to state estimation [45]. The Kalman Filter algorithm manages the state estimate in two stages: predict and update. While the original Kalman Filter works for linear systems, the Extended Kalman Filter (EKF) addresses this shortcoming by linearizing the state transition function f and measurement model h around the current state. It then uses the resulting Jacobian matrices F_t and H_t in place of the original formulation's state transition matrix F and measurement model H , respectively in Eq. (7).

$$\begin{aligned} F_t &= \frac{\partial f}{\partial \hat{\mathbf{x}}_{t-1}} \\ H_t &= \frac{\partial h}{\partial \hat{\mathbf{x}}_{t-1}} \end{aligned} \quad (7)$$

A further extension known as the Unscented Kalman Filter (UKF) eschews the computation of Jacobian matrices in favor of using a set of *sigma points* to carry out state projection [46]. Each sigma point is effectively the most recent system state estimate $\hat{\mathbf{x}}_{t-1}$, but with one dimension perturbed in proportion to the current covariance matrix $\hat{\mathbf{P}}_{t-1}$. Each sigma point is then

¹⁹ Technically, the *tf* library does not permit a given coordinate frame having two parents, so we instead use the map to base_link transform and the inverse of the odom to base_link transform to generate a map to odom transform.

¹⁸ odometry_calibration.xml.

projected through f and the predicted state $\hat{\mathbf{x}}_t$ and covariance $\hat{\mathbf{P}}_t$ are recovered.

In ROS 2, the *robot_localization* (*r_l*) package provides implementations of both an EKF and UKF [47]. The package supports an unlimited number of sensor inputs from common ROS message formats such as Odometry, Imu, PoseWithCovarianceStamped, and TwistWithCovarianceStamped. Users also have the ability to control which dimensions from any given measurement source will be fused in the final state estimate. For example, if an IMU sensor produces faulty heading data due to magnetic interference from the magnetometer, that dimension can be excluded when fusing the IMU measurements. Pose data can also be fused *differentially*, i.e., consecutive poses can be converted into velocities before being passed to the filter. This feature is useful when attempting to fuse world-referenced pose data from sources that do not agree with one another.

r_l uses an omnidirectional three-dimensional kinematic state transition function to estimate the 15-dimensional state vector given in Eq. (8).

$$\mathbf{x} = [x, y, z, \phi, \theta, \psi, \dot{x}, \dot{y}, \dot{z}, \dot{\phi}, \dot{\theta}, \dot{\psi}, \ddot{x}, \ddot{y}, \ddot{z}] \quad (8)$$

While the package does not support any other kinematic models, common models like the unicycle model can be obtained by clamping all three-dimensional state variables (as well as $\dot{y}$ and $\ddot{y}$) to 0. Furthermore, kinematic models such as Ackermann steering can be used if the measurement source provides absolute pose data and the appropriate process noise covariance matrix entries are increased to force the filter to trust measurements more than the filter. In practice, this is sufficient for most mobile robotics applications and the most common method of sensor fusion for state estimation in ROS over the last half-decade.

The large number of parameters available to the user in *r_l* can be used to tune the filter's performance and tailor it to the user's application. In general, the following practices are recommended:

- If operating in a planar environment, set *two_d_mode* to *true*. While the filter will still continue to perform full 3D state estimation, dimensions outside SE2 will be set to 0.
- For differential drive platforms, the only body-frame velocities that are typically relevant are $\dot{x}$ and $\dot{\theta}$. However, the 0 value for $\dot{y}$ that is produced by the wheel encoder odometry should be fused in the filter. This prevents the filter from artificially generating non-zero $\dot{y}$ in the state as the robot goes around turns.
- Every linear and rotational dimension must have a reference (i.e., it must be measured by one of the user's sensor inputs). This measurement can be provided via absolute pose data or velocity data. Prefer pose sources when fusing only a single input in that dimension. Prefer one pose and many velocity sources when fusing multiple sources for a dimension.
- While the filter accepts linear acceleration measurements, they are generally insufficient for state estimation, as the double-integration of linear acceleration without any velocity or pose reference will lead to unbounded growth in that dimension.
- Start with the minimum set required to provide a reference for every linear and angular dimension, and then add inputs one-at-a-time.
- Take time to understand the coordinate frames of sensor data. Make sure that transforms are available for all data that is not provided in the principal coordinate frames.

Fuse The nonlinear state estimation problem can also be solved through iterative numerical optimization. It can be factored into the following least-squares form:

$$\hat{\mathbf{x}}_t = \arg \min_{\mathbf{x}} \sum_i \left(\frac{\mathbf{y}_i - g(\mathbf{x})}{\Sigma^{\frac{1}{2}}} \right)^2 \quad (9)$$

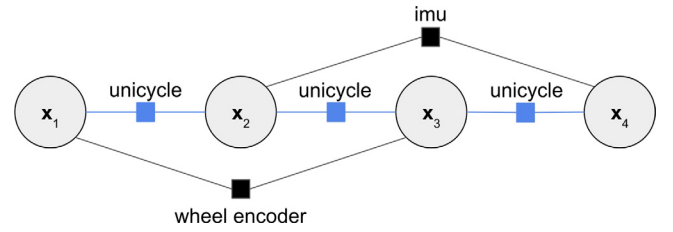


Fig. 15. Example factor graph for a system with asynchronous measurements. A sensor measurement from a wheel encoder creates a constraint between state $\mathbf{x}_1$ and $\mathbf{x}_3$. A measurement from an IMU creates a constraint between state $\mathbf{x}_2$ and $\mathbf{x}_3$. The disjoint graphs are then connected via unicycle model kinematic constraints.

where $\hat{\mathbf{x}}_t$ is the optimal state estimate. We obtain it by finding the state that minimizes the sum squared error between each observation $\mathbf{y}_i$ and the transformed state $g(\mathbf{x})$, weighted by the inverse square root of the covariance Σ .

Factor graphs represent this formulation graphically, as shown in Fig. 15. Two types of nodes are present in the graph: *variable nodes* represent specific quantities that we want to estimate (e.g., the robot's pose at time t) and *factor nodes* represent constraints on those variables (e.g., kinematic constraints via a motion model). In Fig. 15, the variable nodes are the states $\mathbf{x}_1$ through $\mathbf{x}_4$, with the square factor nodes representing constraints between the connected states. This example shows a common use case for sensor fusion, in which asynchronous measurements arrive from different sensors. Each sensor measurement creates a constraint between two states, leading to two separate sets of vertices. In order to unify the graph, we create kinematic constraints between $\mathbf{x}_1$ and $\mathbf{x}_2$, $\mathbf{x}_2$ and $\mathbf{x}_3$, and $\mathbf{x}_3$ and $\mathbf{x}_4$. The graph is then optimized to find the most likely values for the states.

The ROS 2 *fuse* package provides a factor graph framework that can be used in state estimation, mapping, calibration, and more [48,49]. Within *fuse*, each factor node can represent a measurement constraint or a kinematic constraint. Measurement constraints represent the error term in Eq. (9) as $\mathbf{R}^{-\frac{1}{2}}(\mathbf{z}_i - h(\mathbf{x}))$, where h , the sensor model, is a function that transforms the variables in the constraint into the space of the measurement. The covariance matrix $\mathbf{R}$ controls how much weight the error term in question carries during the optimization process. A small covariance would result in the measurement being heavily weighted.

Kinematic constraints represent cost in the form $\mathbf{P}^{-\frac{1}{2}}(\mathbf{x}_i - f(\mathbf{x}_{t-1}))$, where $\mathbf{x}_i$ is the current value of the variables connected to the constraint and the function f is a state transition function that projects the variable values in $\mathbf{x}_{t-1}$ forward in time.

fuse makes extensive use of dynamically loaded plugins to manage the highly configurable graph construction and optimization process. A number of plugins are already provided for common mobile robotics needs, but users may create and utilize their own plugins for custom requirements. Each type of plugin are detailed below.

Variables: Definitions of the quantities to be optimized. A variable may represent a single value, e.g., x position, or it may represent more complex data, such as a timestamped two-dimensional pose. For example, the *Orentation3DStamped* variable comprises a 3D orientation (represented as a quaternion) with an associated time stamp.

Constraints: Connections for variables in the factor graph. Each constraint is responsible for generating some cost given the connected variables. These costs are then minimized during optimization. For example, the *RelativePose2DStampedConstraint* produces a cost for two poses in the graph that is proportional to the difference between those poses and the measured difference as provided by a sensor measurement.

Sensor Models: Models of variables with associated constraints generated from received sensor data. The variables and constraints are then passed to the optimizer. For example, the *Imu2D* sensor model produces the relevant two-dimensional orientation and angular velocity data variables to add to the graph, as well as constraints on those variables as given by the measurement itself.

Motion Models: Models of kinematic constraints between timestamped variables. These are generated on request and passed to the optimizer. For example, the *Unicycle2D* kinematic model generates a constraint between two pose variables in the graph. If those poses are not feasible given a unicycle kinematic model, the constraint will generate non-zero cost.

Optimizers: Build and optimize the graph. The core nodes in *fuse* are effectively wrappers around optimizer plugins. For example, the primary state estimation node in *fuse* is known as the *fixed-lag smoother*. It wraps an optimizer of the same name.

Publishers: Extract relevant quantities from the optimizer and publish the ROS 2 messages containing results. For example, the *Pose2DPublisher* publishes the most recent optimized pose and covariance from the graph as both a *geometry_msgs/PoseStamped* ROS 2 message as well as a transform to be consumed by other nodes via *tf*.

Each iteration, sensor model plugins will receive sensor data and create constraints based on them. The optimizer will receive these constraints and request the motion model plugin to generate constraints to connect the sensor model constraints to the existing graph. These new variables and constraints are optimized over via the optimizer to compute optimal values. Finally, the publisher will return the results over ROS 2 topics to interested nodes.

fuse has a number of advantages over *r_l*, both due to software design and affordances provided by factor graphs:

- *fuse* is more flexible and extensible. In addition to *r_l*'s support of an arbitrary number of sensors, *fuse*'s plugin-based architecture allows configuration of the state vector, motion, and sensor models.
- Relative pose measurements are better supported in *fuse*.
- Linearization errors are reduced in *fuse* at every iteration in the optimization.
- *fuse* can be used for applications outside sensor fusion, mapping, and calibration; one such application is robot control.

However, these features come at an increased compute cost. The experiments in Section 7.2 suggest that *fuse* could require as much as 4x the CPU of *r_l*. As *r_l* is fairly lightweight, this increase is generally acceptable for the vast majority of applications.

For those particularly resource-constrained applications, there are a number of parameters that may be tuned to improve compute time. Reducing the length of the smoothing window of the fixed-lag smoother and the frequency of update can improve the performance. Further, *fuse* uses Google Ceres' automatic differentiation in many of the plugins to compute Jacobians [50]. While this is convenient, it is not as efficient as analytically derived Jacobians, which Ceres also supports.

7.2. Comparison

To compare the two state estimation systems in ROS 2, an experiment was designed and run on a 3.8 GHz AMD Ryzen 5800x CPU. A mobile robot equipped with wheel encoders, an IMU, and a laser scanner was driven along a 541-m long route through a commercial environment to collect a dataset, ending within 10 cm of the starting pose. The odometry and IMU data was collected at 25 Hz. *r_l*'s EKF node and *fuse*'s fixed-lag smoother node each fused linear and angular velocities from the wheel encoders and angular velocities from the IMU. ROS 2's *amcl* package

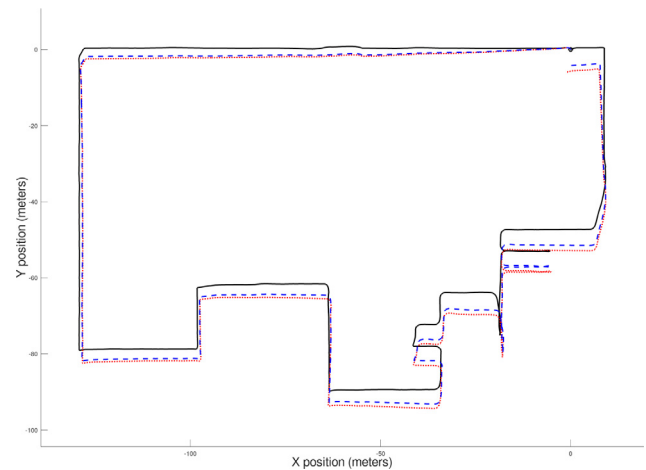


Fig. 16. Three state estimates of the same circuitous path. Black path: *amcl*-generated ground truth path; dotted red path: estimate of the path fusing only wheel encoder and IMU data for *r_l*; dashed blue path: estimate of the path using the same data, but fused via the *fuse* package.

provides the ground-truth information (Section 8.1). The fixed lag smoothing window in *fuse* was set to 0.5 s and the covariance was extracted from the graph at 1 Hz.

The results can be seen in Fig. 16 and Table 5. At the end of the path, the *fuse* estimate was 1.44 m closer to the ground truth position than the *r_l* estimate, though both estimates were well below 5.4 m, or 1%, of the ground-truth pose. The CPU usage for *fuse* was approximately 3.7x that of *r_l*, with a slightly larger standard deviation.

8. Overview of localization and mapping

Localization is the process by which the robot determines where it is with respect to a global frame of reference. As opposed to state estimation, this typically occurs at a reduced rate and provides corrections to odometric drift. A localization system attempts to match its sensor data to a known model of the environment or build the model simultaneously, e.g., Simultaneous Localization and Mapping (SLAM). Commonly, localization is preformed using 2D laser scanners, 3D lidars, or cameras — though techniques exist using more exotic and/or mixtures of sensors. Localization's output is a pose and confidence relative to a world model. This pose solves the REP 105 portion of the *tf* transformation tree from *map* to *odom*.

In ROS 2, the standard localization and SLAM implementations continue to be 2D laser scanner techniques due to their robustness in dynamic and changing environments over long durations. While strides have been made in visual techniques, the field is still rapidly evolving and even relatively mature methods like ORB-SLAM3 are insufficiently robust for large classes of practical robotics tasks [51]. They are generally sufficient for a number of small-scale or short-duration applications, but official ROS 2 support requires further substantive improvements.

8.1. Localization

AMCL One popular approach for solving the localization problem is through the use of particle filters [52], which enable robust state estimation in dynamic environments and in the presence of noisy sensor data. Particle filters represent non-Gaussian statistical distributions using a discrete set of hypotheses (particles). In the case of mobile robot localization, the distribution represents the probable pose of the robot.

Table 5
State estimators comparison.

State estimator	Update frequency (Hz)	Total distance (m)	Total error (m)	Error (% Distance traveled)	CPU %
robot_localization	30	542.16	4.25	0.78	1.38 ± 1.17
fuse	20	542.46	2.81	0.52	5.19 ± 1.37

If an initial pose is not set, a particle filter may initialize the particle cloud by uniformly distributing it throughout the space. Otherwise, the particles are distributed around the provided start pose of the robot. At each update step, the particles are projected using a kinematic model of the robot with noise perturbations. The particles are then weighted based on the likelihood of their sensor observations at that pose. The filter may then resample the particles, that is, generate a new particle distribution by selecting particles from the existing distribution.

The *amcl* (Adaptive Monte Carlo Localization) package contains a pure and fully-parameterized implementation of the grid particle filter localizer, Fig. 17 [53]. The package exposes numerous parameters for controlling the full behavior of the filter: minimum and maximum particle counts, resampling periods, statistical weights, motion model noises, and more. State projection is handled via a motion model that supports differential drive or holonomic bases with configurable characteristics. Particle weight assignment is performed by taking the current laser scan of the environment in the sensor frame, transforming it based on the pose of each particle, and then attempting to match the scan to a rastered map of the environment. Particles that have better matches with the map will therefore have higher weights, and will be more likely to be selected in the next resampling step.

Like the odometry model, the laser model is highly configurable. Two main models are supported: the *beam model* performs ray tracing from each particle to generate a model laser beam, and then compares the ideal beam with the sensed laser measurements [54]. The beam model models four sources of error due to physical phenomenon with the laser scanner and environment: sensor measurement noise, changes in the environment, failures to measure, and random noise. The *likelihood field model* diffuses the map via a Gaussian distribution into a lookup grid such that each cell represents the range from that cell to the nearest occupied cell in the original map [55]. Each laser scan point is then projected onto the lookup grid, and the score for the scan is proportional to the sum of the values of the cells into which the projected points fall. This models sensor noise using the Gaussian distribution, but also explicitly models errors from sensor failures and random measurements in the same way as the beam model. This can be more optimal computationally as well as resolve lack of smoothness concerns with the beam model due to many small, dispersed occupancy grid entries in complex settings. The performance characteristics of the *amcl* package is well researched and can achieve localization accuracies of 5 cm or better in many practical environments [56].

GPS Localization Localizing a robot using GPS sensors is a popular approach for outdoor robot platforms. While subject to drift in some environments, GPS data provides a globally unambiguous positional reference and reduces the need for techniques that rely on expensive sensors. The key to integrating GPS data in a robot's state estimate is the computation of the transform between the robot's world frame (e.g., the map frame as specified in REP-105) and the GPS frame, most commonly the WGS-84 geodetic datum [57].

Generating this transform requires knowledge of the robot's pose in a world frame, the robot's latitude and longitude as provided from a GPS sensor, and the robot's heading in an earth-referenced frame, as might be obtained via an IMU with a magnetometer.



Fig. 17. AMCL's particle cloud localizer in a warehouse environment with a laser scanner. Map in black, laser scan readings in red, particles in green, and the robot's pose as the frame with black circular footprint. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

The *robot_localization* package described in Section 7.1 provides the NavSat Transform Node, whose purpose is to calculate this transformation. It does this by converting the GPS position and earth-referenced heading into a position in the Universal Transverse Mercator (UTM) coordinate frame [58]. From there, the transform to the robot's world frame is trivial. The output of NavSat Transform Node is a pose that can be fused into the robot's state estimate or directly used for localization with respect to the robot's world frame. The accuracy of GPS localization is limited by the environment, quality of the GPS receiver, and whether RTK (Real-time kinematic) GPS corrections are available in the area. When RTK is available, accuracies can reach 2 cm or better [59].

8.2. Mapping

Mapping solves the fundamental problem of how a robot understands its environment. It uses sensor data and potentially robot state estimates to generate a globally accurate model of the world. This world representation is used for many tasks, including global planning and localization, when not using GPS or other global positioning systems. Succinctly, mapping is the process of learning a world's contents from sensor data. For mapping techniques in ROS 2, the map is represented as an occupancy grid, or a grid map of cells representing the probability of occupancy. They also perform mapping and localization simultaneously, e.g. SLAM.

Cartographer provides real-time simultaneous localization and mapping in both 2D and 3D for multiple platforms and sensor configurations using pose-graph optimization [60]. Laser scans are added into submaps over short periods of time to create locally accurate maps with current measurements. Scan matching is performed on these local submaps and the scan is inserted into the submap at its best estimated pose. When a submap is completed, it takes part in a loop closure process which compares submaps and local scans to refine the global occupancy grid model.

While Cartographer is a good option for some users, it is worth noting that out-of-the-box results with Cartographer are typically poor. It is possible to achieve 3–5 cm accuracy using Cartographer, but requires extensive professional tuning using hardware

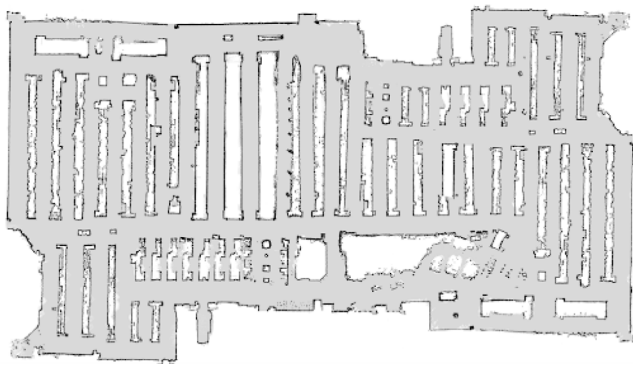


Fig. 18. Map generated by SLAM Toolbox [61].

platforms with high-quality odometry. Further, the original author's organization has halted development and support. ROS 2 support is managed by a community effort but has not received significant attention due to the complexity of the codebase. Thus, it is difficult to recommend the use of Cartographer.

SLAM Toolbox is a set of tools and capabilities for 2D SLAM built upon the OpenKarto SLAM library [62]. SLAM Toolbox is also a pose-graph optimization method. It matches incoming sensor readings with a recent rolling buffer of scans to provide a corrected pose. This uses a correlation grid constructed from the buffer of local measurements to match the incoming scan at a coarse half-resolution, followed up by full resolution for fine tuning the results.

The incoming scan with the initially corrected pose is then evaluated for loop closure. Candidate scans are found which are near the incoming measurement but not connected locally in the graph. The candidates are matched against the scan at a coarse resolution. If the match is strong enough, a full resolution match is performed on the fine correlation grid. If again the match is sufficiently strong, a constraint is added; then the pose-graph optimization process is performed.

Changes were made to the OpenKarto library to increase speed of scan matching using multi-threading and replace the Sparse Bundle Adjustment with Google Ceres to provide more flexible optimization settings. It can reliably map spaces greater than 100,000 sq ft in real-time (Fig. 18), enables serialization of maps for continued mapping, allows for manual manipulation of the pose-graph, and comes with several operating modes.

9. Overview of key utilities

This section provides overviews of new and unique utilities in ROS 2. They contribute intuitive user interactions and create safer robotic systems by commanding, regulating, and validating the autonomous navigation system or its outputs.

Lifecycle Manager governs the state transitions of the nodes. It is used to orchestrate the bringup and shutdown of the system safely and deterministically to guarantee all constituent components of ROS 2 are active and ready before the first command is processed. It also maintains a connection to each server throughout execution to deactivate the system should any component fail. It may also re-activate the system after an error is handled.

Collision Monitor checks for breaches in zones around the robot from sensors such as pointclouds, laser scans, or sonars to prevent imminent collisions, bypassing navigation's planners in case of sudden and urgent changes. These zones may ask a robot to slow to a reduced speed or immediately stop when a sufficient number of returns are detected. These zones may also be scheduled based on velocity. The monitor also has an *Approach*

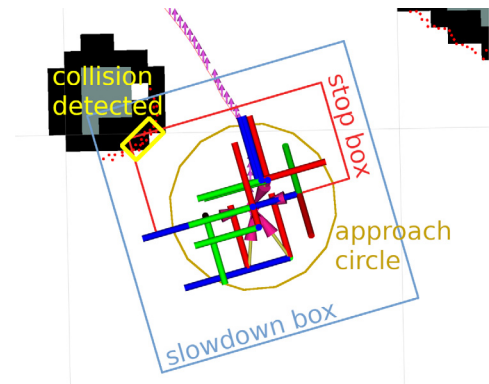


Fig. 19. Collision monitor zones on a sample robot.

mode to project a robot's footprint forward in time and reduce the speed to assure the system is always a certain duration away from potential collisions. An example can be seen in Fig. 19, showing the three primitive types of collision monitoring supported: stop, slowdown, and approach polygons. Note that any convex or concave polygon may be specified in the primary robot body-fixed frame.

Velocity Smoother smooths the output velocities from the navigation system by applying dynamic constraints based on the current robot state. This improves smoothness of the commands sent to a base controller, which can increase the longevity of the hardware. When set to a higher rate than incoming velocity commands, it will also interpolate the commands by dynamic limits providing even smoother output to the hardware at a regular interval.

Simple Commander is a Python3 API to command Nav2 at all levels from the BT Navigator down to its constituent servers allowing a user to have a 'pythonic' user experience. It may be used to create autonomy systems leveraging Nav2 (or bypassing BT Navigator to define custom hand-engineered logic) in Python3. It contains a cost map API for handling incoming occupancy grid messages and querying it for information.

Waypoint Follower is a server used to request navigation to many waypoints and performing some action at each. The waypoint follower uses task executor plugins to complete the action at each waypoint (ex. wait for user input, take a photo, pick up an object, etc.) which are dynamically loaded at run-time and fully configurable. The server also supports looping a series of waypoints for repetitive actions.

Command and Control Panel is an Rviz user interface for interacting with Nav2 for testing, evaluation, or getting started. It provides tools to command the robot with navigate to pose, navigate through poses, and waypoint following tasks. It also visualizes the action feedback, can cancel ongoing tasks, and control the system's lifecycle states.

Robot simulation eases the process of development, testing, and reproduction. Nav2 provides out-of-the-box support for the Gazebo simulator, a sister project of ROS, for testing and evaluation before deploying new capabilities onto hardware robots [63]. Gazebo has been developed and used for over a decade by ROS Community and contains numerous features helpful for building practical robot systems, including:

- Wide variety of supported sensors
- Plugin-based physics with rigid body dynamics
- A client-server architecture allowing distributed computation
- Fully integrated with ROS
- Cross-platform support with cloud integration



Fig. 20. Example of TurtleBot3 Waffle robot, operating in the AWS Bookstore environment simulated by Gazebo [64].

- XML described simulation environments using Simulation Description Files (SDF)

Nav2 provides a set of simulations containing a robot platform in various environments including a sandbox and a warehouse environment for easy testing and evaluation of the framework. As an alternative, additional environments can be easily created for a robot platform, as depicted on Fig. 20.

The Gazebo brand has two distinct products within it: Gazebo (aka Gazebo Classic) and Ignition-Gazebo (aka Ignition). Ignition-Gazebo is a re-write of the legacy software to provide a modular, plugin-based simulation framework. Although not yet as fully featured, Nav2 will soon provide support for Ignition as well [65].

10. The future of ROS 2 mobile robotics

The previous sections discussed the algorithms, behaviors, and features currently available in Nav2, and ROS 2 in general. These have been the product of several years of dedicated collaborations between industry, academia, and individual contributors across the globe. These efforts continue today to develop additional missing technologies for robotics research and commercialization. This section highlights a selection of notable ongoing and future-looking projects in the ROS 2 mobile robotics ecosystem being pursued.

There are many situations where it is necessary to plan global paths based on a reduced representation of the environment, rather than in free-space where the robot may travel anywhere. Large-scale warehouses may be so massive that it is unrealistic to replan entire routes within a reasonable time-frame using free-space planning. They may also require the robot to travel on pre-assigned routes wherever possible. Further, local trajectory planning in non-planar environments (e.g. parking garages, outdoor unstructured environments) may require the use of traversability estimation in the local time-horizon. Rather than creating and storing similar height-maps for global planning, routing is frequently utilized. In development is a Route Server analogous to the Planner Server to enable navigation-graph global route planning to handle all of these situations within the Nav2 framework. It will use a pre-defined graph to plan through spaces with configurable constraints and directionality. The graph may represent a hand-selected set of points, automatically generated nodes and edges from a visibility graph, the result of a Probabilistic Roadmap generator, and more. This will unlock the ability for directional and non-directional lanes or bounded navigation regions, coarse global routing through massive spaces which

may be refined within a window, algorithmically-defined reduced free-space planning, and general outdoor navigation.

Additionally, work is being conducted to extend Nav2's provided environment models out of two-dimensions. Extensions are being designed to facilitate height maps with traversability estimation for outdoor, urban, and natural environments. This will enable new classes of robot systems to leverage Nav2 natively without coarsely approximating terrain information using cost or risk maps. It also improves traditional indoor robot systems, facilitating more accurate ground information to avoid small objects and exploit ramps.

Finally, in the future, more attention will be focused on the problems of two- and three-dimensional localization. AMCL, used commonly in ROS, has a number of shortcomings and the state of the art in localization has improved since its creation. A design has been proposed to create a localization framework for building both 2D and 3D localization systems atop of modular components. In the future, this will allow developers to swap out individual components of standard implementations to build use case-specific or novel localization systems without being required to recreate all other components. Its initial release will contain an analog to AMCL, but also recent algorithms such as DT-NDL-MCL and pose-graph based solutions [66].

11. Conclusion

This paper surveys the state-of-the-art in ROS 2 navigation, as of November 2022, with key metrics and expert insights from principal ROS developers and maintainers. The redesign of Nav2 has triggered a renewed interest in mobile robotics technology in the ROS ecosystem. Since 2018, Nav2 and related projects have continued to develop modern algorithms to augment historically capable methods in continual use since Willow Garage. New global path planning, local trajectory planning, cost map, and state estimation algorithms have been introduced to enhance navigation capabilities to support a broader set of vehicles, environments, and applications. Moreover, entirely new classes of algorithms and systems have been added to hone performance: such as path smoothers, behavior trees, and numerous utilities. This innovation has not terminated — new developments come on a quarterly basis and continues to thrive amongst an ever increasing number of commercial, academic, and individual contributors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

Appendix A. Algorithm selection by robot model

Sections 2–4 compare global path planners, trajectory planners, and smoothers between each other with brief descriptions of their most useful robot model types in Tables 1–3. However, it is useful to provide an expanded discussion on these summaries and provide algorithm optimal use cases, which is further summarized in the Table 6.

Table 6
Summarized algorithm comparison.

Planners	
NavFn	Good general purpose and performant holonomic planner
Theta*	Best at operating in environments with long, straight, and potentially non-axially aligned corridors or open spaces
Smac 2D-A*	Best on circular robots in a heterogeneous fleet containing other non-circular or Ackermann robots for consistent behavior
Smac Hybrid-A*	Best for car-like or Ackermann robots
Smac State Lattice	Best for large non-circular non-Ackermann robots or when requiring a finite set of describable robot motion primitives
Controllers	
DWB	Decent reactive choice when operating in static scenes or far from obstacles, with fully replaceable objective functions
MPPI	Best predictive choice for dynamic scenes and confined areas for all platforms, with fully replaceable objective functions
TEB	Historical predictive choice for dynamic scenes and confined areas for all platforms, with moderately tunable optimization
RPP	Geometric velocity-regulated controller for non-holonomic robots which must follow a path without deviation
Graceful	Good spiral technique for differential-drive robots when movements must be smooth without dense paths
Rotation Shim	Rotation primitive before applying other algorithms to rotate towards path heading, if possible, for smooth path tracking
Smootherers	
Simple Smoother	Good for smoothing holonomic paths to remove defects caused by oscillation and widening out turning maneuvers
Constrained Smoother	Best for feasible paths to guarantee drivability optimizing for cost, curvature, and smoothness – computationally expensive
Savitzky-Golay Smoother	Good for removing localized defects caused by gradient descent, cell binning, or instantaneous changes (NavFn, Theta*)

A.1. Circular differential-drive

Recommendation: NavFn + MPPI

Circular differential-drive robots typically leverage infeasible global path planners. NavFn is the recommended algorithm for such cases, as a high performance and highly deployed implementation used for over a decade. If deploying a circular differential-drive robot in conjunction with other non-circular platforms, the Smac 2D-A* may be a good option to have behavioral consistency with other robots using the feasible Smac Planners – in exchange for increased compute time. When using a robot in non-axially aligned maps, Theta* can be most useful to generate straight-line paths at arbitrary angles.

All of Nav2's trajectory planners are also built with optimizations for circular robots and will work well for this class of robot, but with different behavioral attributes. The use of MPPI or TEB may be beneficial if in highly dynamic settings or where needing to get out of tight spots. These predictive controllers, however, come at an elevated computational cost. Reg. Pure Pursuit is useful for exact-path tracking behavior in static scenes or where deviation from paths is not permissible while having an exceptionally small computational footprint. DWB is a middle-ground all-around option when not in dynamic scenes but deviations from the path would be beneficial – for instance to optimize for smoothness or increased distance from static obstacles than the global infeasible plan provided.

Typically, path smoothers for circular differential drive robots are not necessary for good performance. However, infeasible global plans may be improved with the use of the Savitzky-Golay smoother to remove localized path defects as the result of gradient descent or cell binning, but these changes are largely superficial and do not manifestly impact system behavior when paired with a Nav2-provided trajectory planner.

A.2. Non-circular differential-drive

Recommendation: Smac State Lattice + MPPI + Simple Smoother.

Non-circular differential drive robots generally require the use of feasible planners to guarantee drivability in confined spaces whereas they may not rotate in place without collision. The use of the Smac State Lattice planner is recommended as a high performance option with adjustable motion primitives (including rotating in place). However, Smac Hybrid-A* could also be appropriate if a minimum control set is difficult to model. If non-circular robots are in settings where they may generally rotate in place anywhere they might travel (e.g. not confined or

large), infeasible planners like the NavFn global planner may be deployed instead.

All trajectory planners are built to work with non-circular robots as well, using the robot's footprint to collision check. The same advice applies as described above for circular robots, but with an added computational cost for these heavier collision checking requirements. However, from experience with non-circular robots, it is recommended to leverage the Rotation Shim controller to rotate the platform to the relative path heading before path tracking when using an infeasible planner with either DWB or Graceful. While not required, it is a step-up in an observer's predictability as these may struggle to make massive angular changes effectively.

Providing large, non-circular robots with a smoother global path can impact small oscillatory behaviors notable on non-circular robot platforms due to their shape. Due to this, the simple smoother is recommended as a fast and reasonably effective solution to loosen turns and straighten out oscillations. The constrained smoother may also be applied, but it is computationally intensive and considers more criteria than may be required for a typical differential-drive robot application.

A.3. Omni-directional

Omni-directional robots may follow the same advice outlined in the differential-drive discussions above following the appropriate circular vs. non-circular distinction. The only notable difference is that Reg. Pure Pursuit behavior is not possible for omnidirectional robots to fully utilize its drive-train's lateral movement capabilities. While it may be employed, it will act as though it is a differential drive robot. All other trajectory planners may utilize the full capabilities of omnidirectional platforms.

A.4. Ackermann

Recommendations:

- Smac Hybrid-A* + MPPI,
- Smac Hybrid-A* + RPP + Constrained Smoother

Ackermann robots should typically use the Smac Hybrid-A* planner, a purpose-built planner for Ackermann or car-like robots. However, the State Lattice planner is also applicable if an application demands a finite number of totally predictable maneuvers or behaviors.

It is recommended to use TEB, MPPI or RPP for Ackermann robots. TEB and MPPI provide predictive controllers while RPP provides exact path following behaviors to car-like robots. While

DWB may be applied to Ackermann robots, the trajectory generator for curvature constraints has not yet been contributed to the project.

The constrained smoother is recommended to be paired with the global plan with Ackermann robots to improve drivability and proximity to obstacles. However, this comes at a non-trivial computational overhead and it may only be required for RPP, as MPPI and TEB will internally minimize similar constraints as part of their formulations. The selection of smoother should be made in conjunction with the selection of path tracking behavior (trajectory planner).

Appendix B. Smac Planner framework

The *Smac Planner* within Nav2 is a search-based planning framework with multiple algorithms including Hybrid-A*, State Lattice, and 2D-A*. It contains a shared and highly optimized A* algorithm for which each algorithm utilizes templated by *Node Types*. These node types contain both the node's search information (e.g. visited, queued, cost) but also contains the planner's search characteristics. Specifically, they contain the methods specific to each planner's behaviors such as heuristics, expansion characteristics, and traversal cost functions. All of these functions are shared via static members to gain the performance equivalent of a standalone application. This framework minimizes boilerplate such that new planners may be added in as few as 50 lines of code (with an average of approximately 200).

References

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot Operating System 2: Design, architecture, and uses in the wild, *Science Robotics* 7 (66) (2022) eabm6074.
- [2] S. Macenski, F. Martín, R. White, J.G. Clavero, The marathon 2: A navigation system, in: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, 2020, pp. 2718–2725.
- [3] E. Marder-Eppstein, E. Berger, T. Foote, B. Gerkey, K. Konolige, The office marathon: Robust navigation in an indoor office environment, in: 2010 IEEE International Conference on Robotics and Automation, IEEE, 2010, pp. 300–307.
- [4] M. Jaimez, J.-L. Blanco, J. Gonzalez-Jimenez, Efficient reactive navigation with exact collision determination for 3d robot shapes, *Int. J. Adv. Robot. Syst.* 12 (5) (2015) 63.
- [5] L.C. Wang, L.S. Yong, M. Ang, Hybrid of global path planning and local navigation implemented on a mobile robot in indoor environment, in: Proceedings of the IEEE International Symposium on Intelligent Control, 2002.
- [6] E. Dijkstra, A note on two problems in connexion with graphs, *Numer. Math.* 1 (1959) 269–271.
- [7] S.M. LaValle, Rapidly-Exploring Random Trees : a New Tool for Path Planning, The Annual Research Report, 1998.
- [8] O. Brock, O. Khatib, High-speed navigation using the global dynamic window approach, in: Proceedings 1999 IEEE International Conference on Robotics and Automation (Cat. No.99CH36288C), Vol. 1, 1999.
- [9] A. Nash, K. Daniel, S. Koenig, A. Felner, Theta*: Any-angle path planning on grids, in: AAAI, Vol. 7, 2007, pp. 1177–1183.
- [10] P.E. Hart, N.J. Nilsson, B. Raphael, A formal basis for the heuristic determination of minimum cost paths, *IEEE Trans. Syst. Sci. Cybern.* 4 (2) (1968) 100–107.
- [11] R. Philippsen, A Light Formulation of the E Interpolated Path Replanner, Tech. Rep, ETH Zurich, 2006.
- [12] J.E. Bresenham, Algorithm for computer control of a digital plotter, *IBM Syst. J.* 4 (1) (1965) 25–30.
- [13] A. Nash, S. Koenig, C. Tovey, Lazy Theta*: Any-angle path planning and path length analysis in 3D, in: Proc. AAAI Conf. Artif. Intell., 24, (1) 2010, pp. 147–154.
- [14] D. Dolgov, S. Thrun, M. Montemerlo, J. Diebel, Practical search techniques in path planning for autonomous driving, in: International Symposium on Search Techniques in Artificial Intelligence and Robotics, 2008.
- [15] M. Likhachev, D. Ferguson, Planning long dynamically feasible maneuvers for autonomous vehicles, *Int. J. Robot. Res.* 28 (8) (2009) 933–945.
- [16] M. Pivtoraiko, R.A. Knepper, A. Kelly, Optimal, Smooth, Nonholonomic Mobile Robot Motion Planning in State Lattices, Tech. Rep. CMU-RI-TR-07-15, Robotics Institute, Carnegie Mellon University, Pittsburgh, PA, 2007.
- [17] M. Pivtoraiko, T. Howard, I. Nesnas, A. Kelly, Field experiments in rover navigation via model-based trajectory generation and nonholonomic motion planning in state lattices, in: Proceedings of 9th International Symposium on Artificial Intelligence, Robotics, and Automation in Space (ISAIRAS '08), 2008.
- [18] N. Limpert, S. Schiffer, A. Ferrein, A local planner for Ackermann-driven vehicles in ROS SBPL, in: 2015 Pattern Recognition Association of South Africa and Robotics and Mechatronics International Conference (PRASA-RobMech), 2015, pp. 172–177.
- [19] D. Fox, W. Burgard, S. Thrun, The dynamic window approach to collision avoidance, *Robot. Autom. Mag., IEEE* 4 (1) (1997) 23–33.
- [20] K. Konolige, M. Agrawal, R.C. Bolles, C. Cowan, M. Fischler, B. Gerkey, Outdoor mapping and navigation using stereo vision, in: O. Khatib, V. Kumar, D. Rus (Eds.), Experimental Robotics: The 10th International Symposium on Experimental Robotics, 2008.
- [21] S. Quinlan, O. Khatib, Elastic bands: connecting path planning and control, in: [1993] Proceedings IEEE International Conference on Robotics and Automation, Vol.2, 1993, pp. 802–807.
- [22] C. Rösmann, F. Hoffmann, T. Bertram, Timed-Elastic-Bands for time-optimal point-to-point nonlinear model predictive control, in: 2015 European Control Conference (ECC), 2015, pp. 3352–3357.
- [23] C. Rösmann, Teb_local_planner, 2022, [Online]. Available: https://github.com/rst-tu-dortmund/teb_local_planner.
- [24] R. Kümmerle, G. Grisetti, H. Strasdat, K. Konolige, W. Burgard, G2o: A general framework for graph optimization, in: 2011 IEEE International Conference on Robotics and Automation, 2011, pp. 3607–3613.
- [25] G. Williams, P. Drews, B. Goldfain, J.M. Rehg, E.A. Theodorou, Aggressive driving with model predictive path integral control, in: 2016 IEEE International Conference on Robotics and Automation (ICRA), 2016, pp. 1433–1440.
- [26] R.C. Coulter, Implementation of the pure pursuit path tracking algorithm, 1992.
- [27] L. Kong, Adaptive pure pursuit model for autonomous vehicle path tracking, 2017.
- [28] S. Macenski, S. Singh, F. Martín, J. Ginés Clavero, Regulated pure pursuit for robot path tracking, 2022.
- [29] M. Ferguson, graceful_controller, 2022, [Online]. Available: https://github.com/mikeferguson/graceful_controller.
- [30] J.J. Park, B. Kuipers, A smooth control law for graceful motion of differential wheeled mobile robots in 2D environment, in: 2011 IEEE International Conference on Robotics and Automation, IEEE, 2011, pp. 4896–4902.
- [31] J.M. Snider, Automatic Steering Methods for Autonomous Automobiles, Vol. 6, Tech. Rep. CMU-RI-TR-09-08, Robotics Institute, 2009, p. 2783, no. 61.
- [32] M. Vargovcik, Ceres smoother pull request 2753, 2022, [Online]. Available: <https://github.com/ros-planning/navigation2/pull/2753>.
- [33] A. Savitzky, M.J.E. Golay, Smoothing and differentiation of data by simplified least squares procedures, *Anal. Chem.* 36 (1964) 1627–1639.
- [34] L. Matthies, A. Elfes, Integration of sonar and stereo range data using a grid-based representation, in: Proceedings of the IEEE International Conference on Robotics and Automation (ICRA), 1988, pp. 727–733.
- [35] H.P. Moravec, Sensor fusion in certainty grids for mobile robots, *AI Mag.* 9 (2) (1988) 61–74.
- [36] T. Lozano-Perez, Spatial planning: A configuration space approach, *Comput., IEEE Trans.* 100 (2) (1983) 108–120.
- [37] G.K. Kraetzschmar, G. Pagès Gassull, K. Uhl, Probabilistic quadrees for variable-resolution mapping of large environments, *IFAC Proc. Vol.* 37 (8) (2004) 675–680.
- [38] D.V. Lu, D. Hershberger, W.D. Smart, Layered costmaps for context-sensitive navigation, in: 2014 IEEE/RSJ International Conference on Intelligent Robots and Systems, IEEE, 2014, pp. 709–715.
- [39] K. Museth, J. Lait, J. Johanson, J. Budsberg, R. Henderson, M. Alden, P. Cucka, D. Hill, A. Pearce, OpenVDB: An open-source data structure and toolkit for high-resolution volumes, in: ACM SIGGRAPH 2013 Courses, SIGGRAPH '13, Association for Computing Machinery, 2013.
- [40] S. Macenski, D. Tsai, M. Feinberg, Spatio-temporal voxel layer: A view on robot perception for the dynamic world, *Int. J. Adv. Robot. Syst.* 17 (2) (2020).
- [41] G. Oriolo, G. Ulivi, M. Vendittelli, Fuzzy maps: a new tool for mobile robot perception and planning, *J. Robot. Syst.* 14 (3) (1997) 179–197.
- [42] M. Colledanchise, P. Ögren, Behavior Trees in Robotics and AI: An Introduction, CRC Press, 2018.
- [43] W. Meusens, REP-105: Coordinate frames for mobile platforms, 2010, [Online]. Available: <https://www.ros.org/reps/rep-0105.html>.
- [44] T. Foote, tf: The transform library, in: 2013 IEEE Conference on Technologies for Practical Robot Applications (TePRA), 2013, pp. 1–6.
- [45] R.E. Kalman, A new approach to linear filtering and prediction problems, *Trans. ASME-J. Basic Eng.* 82 (Series D) (1960) 35–45.
- [46] E. Wan, R. Van Der Merwe, The unscented Kalman filter for nonlinear estimation, in: Proceedings of the IEEE 2000 Adaptive Systems for Signal Processing, Communications, and Control Symposium (Cat. No.00EX373), 2000, pp. 153–158.

- [47] T. Moore, D. Stouch, A generalized extended Kalman filter implementation for the robot operating system, in: E. Menegatti, N. Michael, K. Berns, H. Yamaguchi (Eds.), *Intelligent Autonomous Systems 13*, 2016, pp. 335–348.
- [48] S. Williams, Fuse: a Graph-Based Sensor Fusion Framework, Vimeo, 2021, [Online]. Available: <https://vimeo.com/649648867/c524ae57fd>.
- [49] S. Williams, fuse, 2022, [Online]. Available: <https://github.com/locusrobotics/fuse>.
- [50] S. Agarwal, K. Mierle, T.C.S. Team, Ceres solver, 2022, [Online]. Available: <https://github.com/ceres-solver/ceres-solver>.
- [51] A. Merzlyakov, S. Macenski, A comparison of modern general-purpose visual SLAM approaches, in: 2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2021.
- [52] N. Gordon, D. Salmond, A. Smith, Novel approach to nonlinear/non-Gaussian Bayesian state estimation, *IEE Proc. F Radar Signal Process.* UK 140 (2) (1993) 107.
- [53] S. Thrun, W. Burgard, D. Fox, *Probabilistic Robotics (Intelligent Robotics and Autonomous Agents)*, The MIT Press, 2005.
- [54] D. Fox, W. Burgard, S. Thrun, Markov localization for mobile robots in dynamic environments, *J. Artificial Intelligence Res.* 11 (1999) 391–427.
- [55] S. Thrun, A probabilistic on-line mapping algorithm for teams of mobile robots, *Int. J. Robot. Res.* 20 (5) (2001) 335–363.
- [56] G. Vasiljević, D. Miklič, I. Draganjac, Z. Kovačić, P. Lista, High-accuracy vehicle localization for autonomous warehousing, *Robot. Comput.-Integr. Manuf.* 42 (2016) 1–16.
- [57] N. Imagery, M. Agency, United States Department of Defense, World Geodetic System 1984: Its Definition and Relationships with Local Geodetic Systems, Tech. Rep., TR8350.2, 2000.
- [58] M. Buchroithner, R. Pfahlbusch, Geodetic grids in authoritative maps – new findings about the origin of the UTM grid, *Cartogr. Geogr. Inf. Sci.* (2016) 1–15.
- [59] A.n. Pirtti, N. Arslan, B. Deveci, O. Aydin, H. Erkaya, R. Hoşbaş, Real-time kinematic GPS for cadastral surveying, *Surv. Rev. - SURV REV* 41 (2009) 339–351.
- [60] W. Hess, D. Kohler, H. Rapp, D. Andor, Real-time loop closure in 2D LIDAR SLAM, in: 2016 IEEE International Conference on Robotics and Automation (ICRA), 2016, pp. 1271–1278.
- [61] S. Macenski, On use of the slam toolbox: A fresh (er) look at mapping and localization for the dynamic world, in: *ROSCON*, 2019.
- [62] S. Macenski, I. Jambrecic, SLAM Toolbox: SLAM for the dynamic world, *J. Open Source Softw.* 6 (61) (2021) 2783.
- [63] N. Koenig, A. Howard, Design and use paradigms for Gazebo, an open-source multi-robot simulator, in: 2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No.04CH37566), Vol. 3, 2004, pp. 2149–2154, vol.3.
- [64] AWS Bookstore Gazebo world, [Online]. Available: <https://github.com/aws-robotics/aws-robomaker-bookstore-world>.
- [65] Gazebo Classic vs. Ignition feature comparison, 2022, [Online]. Available: <https://gazebosim.org/docs/garden/comparison>.
- [66] R. Valencia, J. Saarinen, H. Andreasson, J. Vallvé, J. Andrade-Cetto, A.J. Lilienthal, Localization in highly dynamic environments using dual-timescale NDT-MCL, in: 2014 IEEE International Conference on Robotics and Automation (ICRA), 2014, pp. 3956–3962.



Steve Macenski is the Senior Technical Lead at Samsung Research, where he consults for robotics groups in fundamental research, applied research, and commercialization throughout Samsung's global subsidiaries. He is responsible for formulating Samsung's strategic interests in robotics open-source technology and helps Samsung's teams make better use of openly available technology in a number of domains to accelerate their work. He leads distributed engineering teams on high-impact, high-visibility problems facing the commercialization of autonomous systems across

many industries.

He is a core leader in the Robot Operating System (ROS) developing and maintaining 50+ packages spanning nearly every domain in mobile robotics and machine perception. He is a member of the Open-Source Robotics Foundation's ROS Technical Steering Committee (TSC) comprising of industry leaders in the development and proliferation of robotics technology. He also leads ROS 2's autonomous navigation framework and system, Nav2, which is used by over 100+ companies including several in the Fortune 500.

Steve holds a degree in Aerospace Engineering from the University of Illinois at Urbana-Champaign.